Chapter 1

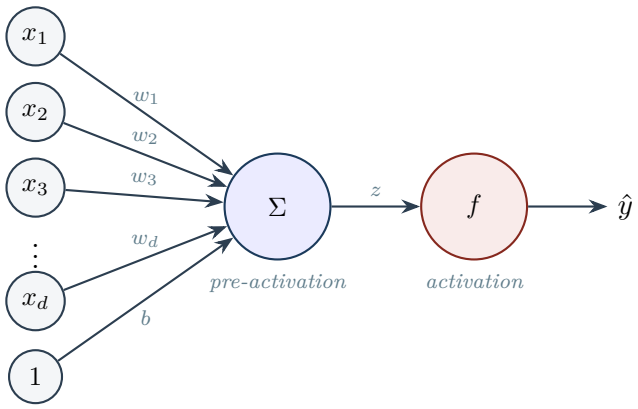
The Anatomy of a Single Unit

Perceptrons and the Mechanics of Learning

1 The Computational Neuron

If we strip away the biological folklore surrounding artificial intelligence, a neural network is fundamentally a *parameterized directed acyclic graph* that transforms input numbers into output numbers. At its bedrock sits the simplest functional unit: the **Perceptron**.

Let us define a single data point $\mathbf{x} \in \mathbb{R}^d$ and an associated binary target y. In standard machine learning notation, we often introduce a bias term $b \in \mathbb{R}$ directly alongside our weight vector $\mathbf{w} \in \mathbb{R}^d$. The computation inside this single unit decomposes strictly into two stages: an *affine transformation* (the pre-activation) followed by a *non-linear mapping* (the activation).



The pre-activation scalar z is the dot product of the input features and their associated weights, offset by the bias:

$$z = \sum_{i=1}^{d} w_i x_i + b = \mathbf{w}^\top \mathbf{x} + b. \tag{1}$$

In Frank Rosenblatt's classical formulation (1958), the activation function is an uncompromising step function:

$$f(z) = \begin{cases} 1 & \text{if } z \geq 0, \\ 0 & \text{if } z < 0. \end{cases} \tag{2}$$

Step activation and a single prediction

```python
import numpy as np

def step(z: float) -> int:
    return 1 if z >= 0 else 0

def predict_single(x: np.ndarray, w: np.ndarray, b: float) -> int:
    z = np.dot(w, x) + b
    return step(z)
```

The weights w_i govern the relative leverage of each incoming feature. If x_1 represents a candidate's CGPA and x_2 represents their resume score, the ratio $|w_1|/|w_2|$ conveys the relative linear importance the model assigns to academic standing over presentation polish. The bias b sets the baseline threshold: how inclined the unit is to fire when all inputs are precisely zero.

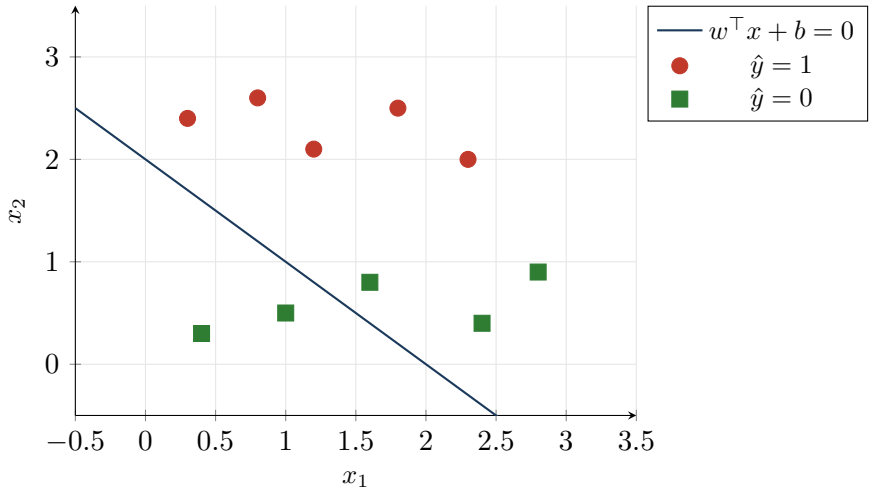
2 The Geometric Perspective: Separating Hyperplanes

In two dimensions ($d = 2$), the equation dividing the decisions—where $z = 0$—defines a straight line:

$$w_1 x_1 + w_2 x_2 + b = 0 \quad \Longrightarrow \quad x_2 = -\frac{w_1}{w_2} x_1 - \frac{b}{w_2}. \tag{3}$$

Every point $\mathbf{x}$ in the feature space falls on one side of this boundary or the other:

- Points evaluated as $\mathbf{w}^\top \mathbf{x} + b \geq 0$ define the **positive half-space** ($\hat{y} = 1$).
- Points evaluated as $\mathbf{w}^\top \mathbf{x} + b < 0$ define the **negative half-space** ($\hat{y} = 0$).



In three dimensions, this dividing boundary is a flat plane ($w_1 x_1 + w_2 x_2 + w_3 x_3 + b = 0$). In arbitrary d-dimensional space, it is a $(d-1)$-dimensional **hyperplane**.

Key geometric fact

Because the decision boundary is strictly planar, a single perceptron can cleanly divide classes **if and only if the underlying data points are linearly separable**.

3 The Perceptron Trick: Heuristic Parameter Updates

Before formulating calculus-based loss minimization, consider Rosenblatt's original update rule: the geometric heuristic commonly termed the *Perceptron Trick*.

Suppose we start with arbitrary initial weights $\mathbf{w}$ and bias b. We sample an observation $(\mathbf{x}_j, y_j)$. The model computes $\hat{y}_j = \text{step}(\mathbf{w}^\top \mathbf{x}_j + b)$. If $\hat{y}_j = y_j$, the point is satisfied; we do nothing.

If the point is misclassified, the decision boundary lies on the wrong side of $\mathbf{x}_j$. To pull the boundary toward the misclassified sample, we nudge the parameters along the direction of $\mathbf{x}_j$ proportional to the sign of the error:

$$\mathbf{w} \leftarrow \mathbf{w} + \eta\,(y_j - \hat{y}_j)\,\mathbf{x}_j, \tag{4}$$

$$b \leftarrow b + \eta\,(y_j - \hat{y}_j). \tag{5}$$

Here, $\eta \in (0, 1]$ represents the *learning rate*—a damping factor to prevent erratic, oscillatory overcorrections.

Vectorized NumPy Implementation

By prepending a constant 1 to our input vector $\mathbf{x}$ (the *bias trick*), we fold the scalar b directly into an augmented parameter vector $\tilde{\mathbf{w}} = [b,\ w_1,\ w_2,\ \ldots,\ w_d]^\top$ and input $\tilde{\mathbf{x}} = [1,\ x_1,\ x_2,\ \ldots,\ x_d]^\top$:

Perceptron heuristic (vectorized)

```python
import numpy as np

def train_perceptron_heuristic(X: np.ndarray, y: np.ndarray,
                               epochs: int = 1000, lr: float = 0.1)
    :
    N, d = X.shape
    # Augment input with a leading column of ones
    X_aug = np.insert(X, 0, 1.0, axis=1)
    # Initialize all weights and bias to 1.0
    weights = np.ones(d + 1)

    for _ in range(epochs):
        j = np.random.randint(0, N)
        x_j = X_aug[j]
        y_true = y[j]

        # Forward pass
        z = np.dot(x_j, weights)
        y_hat = 1 if z >= 0 else 0

        # Parameter update
        error = y_true - y_hat
        weights += lr * error * x_j

    b = weights[0]
    w = weights[1:]
    return w, b
```

4 The Limits of the Heuristic: Why Loss Functions Matter

The heuristic update algorithm works reasonably well on synthetic, well-behaved toy examples, but it suffers from severe theoretical and practical deficiencies:

1. **Non-Uniqueness and Margin Blindness.** For any linearly separable dataset, infinitely many hyperplanes can achieve zero classification error. The Perceptron heuristic terminates as soon as all training samples happen to fall on the correct side, offering no mechanism to prefer a robust, margin-maximizing separator over a fragile boundary skimming microns away from a training point.

2. **Pathological Stalling.** Because updates are triggered only by zero-one step errors, points that are technically misclassified but very close to the boundary carry the exact same mathematical weight as points placed deep in opposing territory.

3. **The Non-Differentiability Barrier.** The step activation $f(z)$ has a derivative of zero almost everywhere ($\frac{df}{dz} = 0$ for $z \neq 0$) and is undefined at $z = 0$. You cannot apply classical gradient-based optimization through a step function: the local slope tells you nothing about which direction reduces error.

The barrier to modern architectures

To build modern, scalable architectures, we must ditch the heuristic step function, adopt smooth or piece-wise differentiable surrogates, and define explicit, quantitative **loss functions**.

5 Differentiable Formulations: Perceptron Loss and Hinge Loss

To quantify margin violations systematically, let targets be labeled symmetrically: $y \in \{-1, +1\}$.

The product $y_i\,(\mathbf{w}^\top \mathbf{x}_i + b)$ acts as the *signed margin*:

- If $y_i\,(\mathbf{w}^\top \mathbf{x}_i + b) > 0$, the classification is correct.
- If $y_i\,(\mathbf{w}^\top \mathbf{x}_i + b) < 0$, the observation is misclassified.

The Classical Perceptron Criterion

The original Perceptron loss penalizes only points that are misclassified, measured by their unnormalized projection distance to the boundary:

$$L_{\text{perceptron}}(\mathbf{w}, b) \;=\; \sum_{i \in \mathcal{M}} -y_i\,(\mathbf{w}^\top \mathbf{x}_i + b) \;=\; \sum_{i=1}^{N} \max\!\big(0,\; -y_i\,(\mathbf{w}^\top \mathbf{x}_i + b)\big), \tag{6}$$

where $\mathcal{M}$ is the set of indices where $y_i\,(\mathbf{w}^\top \mathbf{x}_i + b) < 0$.

The Subgradient

For any misclassified sample $i \in \mathcal{M}$:

$$\nabla_{\mathbf{w}} L_i = -y_i\,\mathbf{x}_i, \qquad \frac{\partial L_i}{\partial b} = -y_i. \tag{7}$$

Applying gradient descent ($\mathbf{w} \leftarrow \mathbf{w} - \eta\,\nabla_{\mathbf{w}} L_i$) yields:

$$\mathbf{w} \leftarrow \mathbf{w} + \eta\,y_i\,\mathbf{x}_i, \tag{8}$$

$$b \leftarrow b + \eta\,y_i. \tag{9}$$

This recovers the Perceptron update cleanly via gradient descent on a surrogate loss:

Perceptron loss by gradient descent

```python
def train_perceptron_loss(X: np.ndarray, y: np.ndarray,
                          epochs: int = 1000, lr: float = 0.1):
    # Assumes y in {-1, +1}
    N, d = X.shape
    w = np.ones(d)
    b = 1.0

    for _ in range(epochs):
        for i in range(N):
            margin = (np.dot(w, X[i]) + b) * y[i]
            if margin < 0:
                w += lr * y[i] * X[i]
                b += lr * y[i]

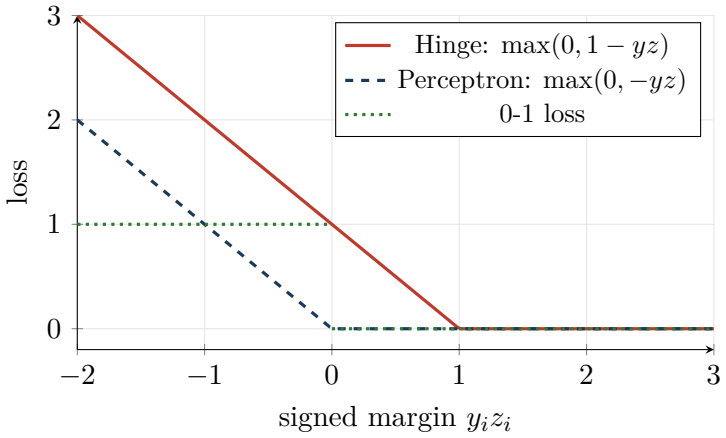
    return w, b
```

The Hinge Loss (Linear SVM)

If we enforce an explicit safety margin of distance 1 away from the boundary, we arrive at the **Hinge Loss**:

$$L_{\text{hinge}}(\mathbf{w}, b) = \frac{1}{N}\sum_{i=1}^{N} \max(0,\ 1 - y_i(\mathbf{w}^\top \mathbf{x}_i + b)). \tag{10}$$

Here, the loss penalizes not only outright misclassifications ($y_i z_i < 0$), but also samples that are technically correct but precariously close to the boundary ($0 \leq y_i z_i < 1$).

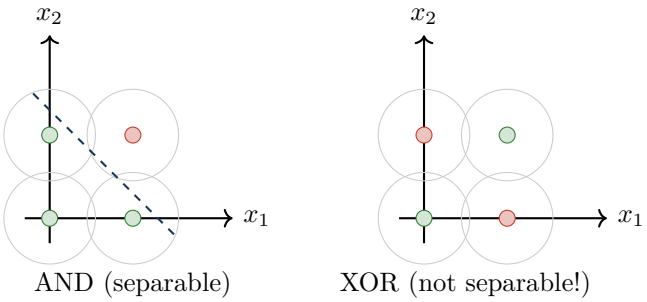


6 The Failure Mode: The XOR Breakdown

In 1969, Marvin Minsky and Seymour Papert published *Perceptrons*, formalizing an elementary mathematical limitation that brought an abrupt halt to neural network research funding—the onset of the first "AI Winter".

Consider the simplest logical boolean operators across two inputs $x_1, x_2 \in \{0, 1\}$:

x_1	x_2	AND	OR	XOR
0	0	0	0	**0**
0	1	0	1	**1**
1	0	0	1	**1**
1	1	1	1	**0**



While AND and OR can be cleaved apart by a linear separator, XOR fundamentally cannot.

Mathematical Impossibility Proof

Assume there exist real weights w_1, w_2 and bias b that satisfy the XOR truth table under a threshold activation:

$$\text{For } (0,0) \to 0: \quad w_1(0) + w_2(0) + b < 0 \implies b < 0. \tag{1}$$

$$\text{For } (0,1) \to 1: \quad w_1(0) + w_2(1) + b \geq 0 \implies w_2 + b \geq 0. \tag{2}$$

$$\text{For } (1,0) \to 1: \quad w_1(1) + w_2(0) + b \geq 0 \implies w_1 + b \geq 0. \tag{3}$$

$$\text{For } (1,1) \to 0: \quad w_1(1) + w_2(1) + b < 0 \implies w_1 + w_2 + b < 0. \tag{4}$$

Sum conditions (2) and (3):

$$(w_1 + b) + (w_2 + b) \geq 0 \implies w_1 + w_2 + 2b \geq 0. \tag{11}$$

Substitute condition (1) ($b < 0$): since $b < 0$, it must be that $w_1 + w_2 + b > w_1 + w_2 + 2b \geq 0$, which implies:

$$w_1 + w_2 + b > 0. \tag{12}$$

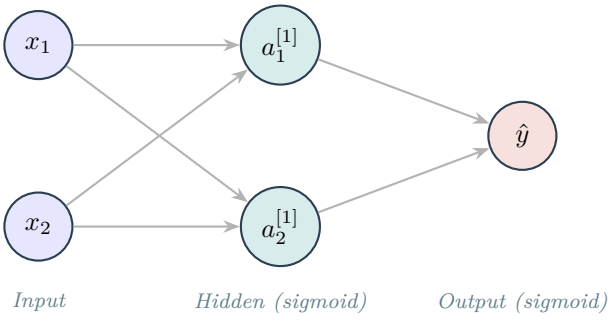
This directly contradicts condition (4), which requires $w_1 + w_2 + b < 0$. No real-valued tuple (w_1, w_2, b) can ever satisfy this system.

The way forward

A single linear hyperplane cannot untangle non-linear topological spaces. To solve XOR, we must compose multiple units into hierarchical layers, folding and stretching the representation space until the classes become linearly separable.

7 Building a Complete Multi-Layer Perceptron From Scratch

Let us construct a Multi-Layer Perceptron (MLP) for binary classification with one hidden layer. We will discard all deep learning libraries and build the network using pure Python and vector arithmetic.



Network Dimensions and Forward Pass

$$\begin{aligned}
\text{Input: } & \mathbf{x} \in \mathbb{R}^{2\times 1}. \\
\text{Hidden Layer: } & \mathbf{W}^{[1]} \in \mathbb{R}^{2\times 2},\ \mathbf{b}^{[1]} \in \mathbb{R}^{2\times 1}. \\
\text{Pre-activation 1: } & \mathbf{z}^{[1]} = (\mathbf{W}^{[1]})^\top \mathbf{x} + \mathbf{b}^{[1]}. \\
\text{Post-activation 1: } & \mathbf{a}^{[1]} = \sigma(\mathbf{z}^{[1]}). \\
\text{Output Layer: } & \mathbf{W}^{[2]} \in \mathbb{R}^{2\times 1},\ b^{[2]} \in \mathbb{R}. \\
\text{Pre-activation 2: } & z^{[2]} = (\mathbf{W}^{[2]})^\top \mathbf{a}^{[1]} + b^{[2]}. \\
\text{Prediction: } & \hat{y} = a^{[2]} = \sigma(z^{[2]}),
\end{aligned}$$

where $\sigma(z) = \dfrac{1}{1+e^{-z}}$.

The Loss Function: Binary Cross-Entropy

For target $y \in \{0, 1\}$, the Binary Cross-Entropy (BCE) loss for a single observation is:

$$L(y, \hat{y}) = -\big[\, y \ln(\hat{y}) + (1-y)\ln(1-\hat{y}) \,\big]. \tag{13}$$

Exact Gradient Derivation via the Chain Rule

Recall that the derivative of the sigmoid function satisfies:

$$\frac{d\sigma(z)}{dz} = \sigma(z)(1-\sigma(z)) = a(1-a). \tag{14}$$

For the output unit:

$$\frac{\partial L}{\partial \hat{y}} = -\frac{y}{\hat{y}} + \frac{1-y}{1-\hat{y}} = \frac{\hat{y}-y}{\hat{y}(1-\hat{y})}. \tag{15}$$

Applying the chain rule through the output activation:

$$\delta^{[2]} = \frac{\partial L}{\partial z^{[2]}} = \frac{\partial L}{\partial \hat{y}} \cdot \frac{d\hat{y}}{dz^{[2]}} = \frac{\hat{y}-y}{\hat{y}(1-\hat{y})} \cdot \hat{y}(1-\hat{y}) = \hat{y} - y. \tag{16}$$

The output gradients follow immediately:

$$\frac{\partial L}{\partial \mathbf{W}^{[2]}} = \mathbf{a}^{[1]}(\delta^{[2]})^\top, \qquad \frac{\partial L}{\partial b^{[2]}} = \delta^{[2]}. \tag{17}$$

Backpropagating to the hidden layer:

$$\frac{\partial L}{\partial \mathbf{a}^{[1]}} = \mathbf{W}^{[2]}\delta^{[2]}, \tag{18}$$

$$\delta^{[1]} = \frac{\partial L}{\partial \mathbf{z}^{[1]}} = \frac{\partial L}{\partial \mathbf{a}^{[1]}} \odot \sigma'(\mathbf{z}^{[1]}) = (\mathbf{W}^{[2]}\delta^{[2]}) \odot \mathbf{a}^{[1]} \odot (1 - \mathbf{a}^{[1]}). \tag{19}$$

And the parameter gradients for the hidden layer:

$$\frac{\partial L}{\partial \mathbf{W}^{[1]}} = \mathbf{x}(\delta^{[1]})^\top, \qquad \frac{\partial L}{\partial \mathbf{b}^{[1]}} = \delta^{[1]}. \tag{20}$$

MinimalMLP: a 2-layer network from scratch

```python
import numpy as np

class MinimalMLP:
    """
    A 2-Layer Neural Network (2 inputs -> 2 hidden units -> 1 output)
    implemented with explicit, non-framework forward and backward passes.
    """
    def __init__(self, seed: int = 42):
        np.random.seed(seed)
        # Weights connecting input to hidden layer (2, 2)
        self.W1 = np.random.randn(2, 2) * 0.5
        self.b1 = np.zeros((2, 1))

        # Weights connecting hidden to output layer (2, 1)
        self.W2 = np.random.randn(2, 1) * 0.5
        self.b2 = np.zeros((1, 1))

    @staticmethod
    def sigmoid(z: np.ndarray) -> np.ndarray:
        return 1.0 / (1.0 + np.exp(-np.clip(z, -30, 30)))

    def forward(self, x: np.ndarray):
        # x is assumed to be shape (2, 1)
        self.x = x
        self.z1 = np.dot(self.W1.T, self.x) + self.b1    # (2, 1)
        self.a1 = self.sigmoid(self.z1)                  # (2, 1)

        self.z2 = np.dot(self.W2.T, self.a1) + self.b2   # (1, 1)
        self.a2 = self.sigmoid(self.z2)                  # (1, 1)
        return self.a2

    def backward(self, y: float, lr: float = 0.1):
        y_hat = self.a2[0, 0]

        # Output error delta: dL/dz2 = y_hat - y
        delta2 = y_hat - y                                # Scalar

        # Gradients for layer 2
        dW2 = self.a1 * delta2                            # (2, 1)
        db2 = delta2                                      # Scalar

        # Propagate error backwards to hidden layer
        delta1 = (self.W2 * delta2) * self.a1 * (1.0 - self.a1)  # (2, 1)

        # Gradients for layer 1
        dW1 = np.dot(self.x, delta1.T)                    # (2, 2)
        db1 = delta1                                      # (2, 1)

        # Stochastic Gradient Descent updates
        self.W2 -= lr * dW2
        self.b2 -= lr * db2
        self.W1 -= lr * dW1
```

```
        self.b1 -= lr * db1

        # Binary Cross-Entropy Loss
        eps = 1e-12
        loss = -(y * np.log(y_hat + eps) + (1.0 - y) * np.log(1.0 - y_hat + eps))
        return loss

# Solving the non-linear XOR problem
if __name__ == "__main__":
    X = np.array([[0.0, 0.0],
                  [0.0, 1.0],
                  [1.0, 0.0],
                  [1.0, 1.0]])
    y = np.array([0.0, 1.0, 1.0, 0.0])

    net = MinimalMLP(seed=1337)

    for epoch in range(10000):
        epoch_loss = 0.0
        for i in range(len(X)):
            x_i = X[i].reshape(2, 1)
            y_i = y[i]
            net.forward(x_i)
            loss = net.backward(y_i, lr=0.5)
            epoch_loss += loss

        if (epoch + 1) % 2000 == 0:
            print(f"Epoch {epoch+1:5d} | Loss: {epoch_loss / len(X):.5f}")

    print("\n--- Final Evaluations ---")
    for i in range(len(X)):
        x_i = X[i].reshape(2, 1)
        pred = net.forward(x_i)[0, 0]
        print(f"Input: {X[i]} | True Target: {y[i]} | "
              f"Predicted: {pred:.4f} (Round -> {round(pred)})")
```

When you run this optimization routine, the network drives its cross-entropy loss down and converges to the correct solution for XOR. The hidden layer takes the two non-separable input dimensions, transforms them non-linearly, and projects them into an intermediate 2D representation where the output perceptron can cut them in two with a single linear decision line.

Exercises

1. **Analytical Subgradient.** Derive the subgradient of the Hinge loss $L(\mathbf{w}, b) = \max(0, 1 - y_i(\mathbf{w}^\top \mathbf{x}_i + b))$ with respect to $\mathbf{w}$ for the case where $y_i(\mathbf{w}^\top \mathbf{x}_i + b) = 1$. Why does this boundary point present an issue for classical gradient descent, and how does backpropagation resolve it?

2. **Failure of Pure Linearity.** Suppose we drop the non-linear sigmoid activation from the hidden layer of `MinimalMLP`, defining $\mathbf{a}^{[1]} = \mathbf{z}^{[1]}$. Prove algebraically that the resulting two-layer network collapses to an ordinary single-layer affine model, demonstrating why deep architectures require non-linearities between layers.

3. **Loss Derivation for Mean Squared Error (MSE).** Suppose we adapt `MinimalMLP` to continuous regression targets $y \in \mathbb{R}$ by using an identity output activation ($a^{[2]} = z^{[2]}$) and replacing BCE with the squared loss $L = \frac{1}{2}(y - \hat{y})^2$. Compute $\delta^{[2]} = \frac{\partial L}{\partial z^{[2]}}$ and trace out the new parameter update equations for $\mathbf{W}^{[2]}$, $b^{[2]}$, $\mathbf{W}^{[1]}$, and $\mathbf{b}^{[1]}$.

Chapter 2

The Training Dynamics

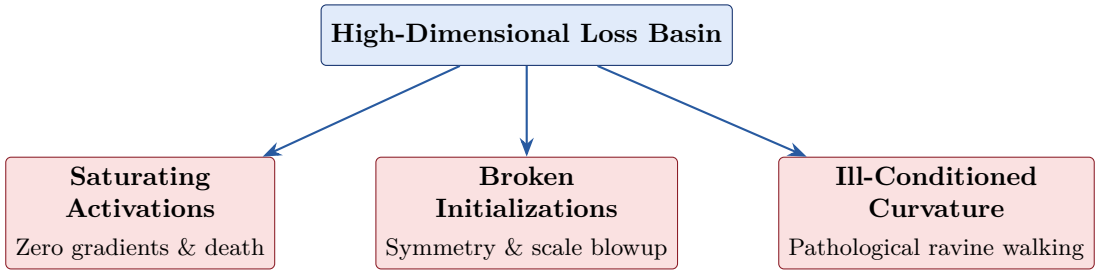
Activations, Initialization, Normalization, and Optimization

---

1 The Landscape of Training Difficulties

Training an Artificial Neural Network is fundamentally an empirical physics simulation: you set up a high-dimensional loss surface, place a parameter state vector onto it, and let numerical mechanics roll it toward a valid minimizer. In modern architectures, that landscape is non-convex, corrugated by steep cliffs, broad plateaus, and degenerate valleys.

High-Dimensional Loss Basin



The fundamental challenges encountered during iterative training can be organized into three structural bottlenecks:

1. **Gradient Degradation (Vanishing & Exploding):** When gradients flow backward through deep compositions of operators, scalar factors multiply repeatedly. If intermediate local Jacobians have singular values strictly bounded below 1, feedback signals vanish exponentially before reaching early layers. If they dominate above 1, updates diverge uncontrollably into numerical overflow.

2. **Ill-Conditioned Curvature and Anisotropic Slopes:** An optimization surface frequently possesses vastly unequal curvatures along different coordinate directions (e.g., when inputs inhabit radically different numerical scales). The gradient vector points almost orthogonally to the direction of the true minimizer, driving standard gradient descent into violent lateral oscillations while creeping agonizingly slowly along the progress axis.

3. **Internal Representation Drift:** As upstream parameter matrices update, the distributions of intermediate activations shift continuously across training steps—a moving-target effect historically termed *internal covariate shift*. This instability forces downstream layers to perpetually readapt to non-stationary distributions, restricting safe learning rates to small values.

Roadmap

To train deep models reliably, we systematically address each breakdown point: **nonlinear transfer mechanics**, **initial weight variances**, **internal distribution stabilizers**, and **adaptive moment optimizers**.

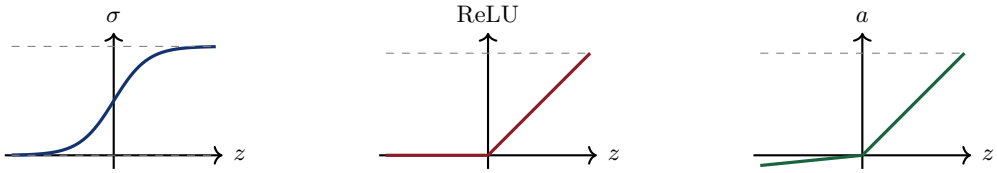
2 Activation Functions: Signal Flow and the Dying Gradient

An activation function transforms raw affine sums $z = \mathbf{w}^T\mathbf{x} + b$ into representational states. Without non-linearities, any arbitrary depth L collapses algebraically into a single linear map:

$$\mathbf{a}^{[L]} = \mathbf{W}^{[L]} \dots (\mathbf{W}^{[2]}(\mathbf{W}^{[1]}\mathbf{x} + \mathbf{b}^{[1]}) + \mathbf{b}^{[2]}) + \mathbf{b}^{[L]} = \tilde{\mathbf{W}}\mathbf{x} + \tilde{\mathbf{b}} \tag{1}$$

However, the specific mathematical properties of that non-linearity determine whether gradient signals survive backpropagation.

Activation Function Geometries



Sigmoid / Tanh
Saturating
Vanishing gradients

ReLU
One-sided hard zero
Dying units

Leaky ReLU / ELU
Continuous floor
Robust gradient pass

2.1 The Sigmoid Function and its Pathologies

The standard logistic sigmoid maps real numbers into bounded probabilities:

$$\sigma(z) = \frac{1}{1 + e^{-z}} \tag{2}$$

Its first derivative exhibits an analytical form:

$$\frac{d\sigma(z)}{dz} = \sigma(z)(1 - \sigma(z)) \tag{3}$$

The maximum value of $\sigma'(z)$ occurs precisely at $z = 0$, where $\sigma(0) = 0.5$, yielding $\sigma'(0) = 0.25$. For inputs $|z| > 4$, the output squashes tightly toward 0 or 1, causing the local derivative $\sigma'(z)$ to decay directly toward zero. Because backpropagation chains these factors layer by layer:

$$\frac{\partial \mathcal{L}}{\partial \mathbf{z}^{[1]}} = \left(\prod_{l=2}^{L} \mathbf{W}^{[l]T} \operatorname{diag}(\sigma'(\mathbf{z}^{[l]})) \right) \frac{\partial \mathcal{L}}{\partial \mathbf{z}^{[L]}} \tag{4}$$

Every sigmoid layer introduces a scaling multiplier of at most 0.25. In an 8-layer network, the upstream gradient is scaled down by a factor of $(0.25)^8 \approx 1.5 \times 10^{-5}$, halting learning in early layers.

Non-Zero-Centering Pathology

Because $\sigma(z) > 0$ strictly for all $z \in \mathbb{R}$, intermediate outputs are entirely non-zero-centered. For an arbitrary downstream node j:

$$\frac{\partial \mathcal{L}}{\partial w_i} = \frac{\partial \mathcal{L}}{\partial z} \cdot x_i = \delta \cdot a_i \tag{5}$$

If every input activation a_i is strictly positive, all weight gradients $\frac{\partial \mathcal{L}}{\partial w_i}$ associated with that neuron take the identical sign of the scalar backpropagated error δ. The parameter vector $\mathbf{w}$ cannot update its coordinates independently; it is restricted to moving along the all-positive or all-negative orthants, generating inefficient, slow convergence.

2.2 The Hyperbolic Tangent Function

The tanh function addresses the zero-centering problem:

$$\tanh(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}} \in (-1, 1), \qquad \frac{d\tanh(z)}{dz} = 1 - \tanh^2(z) \tag{6}$$

Because the output is symmetrically distributed around zero ($\mathbb{E}[\tanh(z)] \approx 0$), incoming gradients can alternate signs freely across dimensions. However, the peak derivative is strictly bounded by 1.0 at $z = 0$, and saturates rapidly as $|z|$ grows, leaving the vanishing gradient problem intact for large pre-activations.

2.3 The Rectified Linear Unit (ReLU) and the "Dying ReLU" Phenomenon

The Rectified Linear Unit eliminates saturation over the positive domain:

$$\text{ReLU}(z) = \max(0, z), \qquad \frac{d\,\text{ReLU}(z)}{dz} = \begin{cases} 1 & \text{if } z > 0 \\ 0 & \text{if } z < 0 \end{cases} \tag{7}$$

Because its derivative is exactly 1 whenever $z > 0$, it does not shrink gradients along positive pathways, preserving deep signal propagation.

The Dying ReLU Problem

Suppose an aggressive parameter update (driven by an overly large learning rate η) or a large negative bias pulls the pre-activation negative across all dataset samples:

$$z = \mathbf{w}^T\mathbf{x} + b < 0 \quad \forall \mathbf{x} \in \mathcal{D} \tag{8}$$

For every input, the activation evaluates to 0, and its derivative evaluates to 0. The gradient flowing through this neuron drops to zero:

$$\frac{\partial \mathcal{L}}{\partial \mathbf{w}} = \delta \cdot \mathbf{x} = 0 \cdot \mathbf{x} = \mathbf{0}, \qquad \frac{\partial \mathcal{L}}{\partial b} = 0 \tag{9}$$

The weights and bias freeze permanently. The unit no longer learns, transforming into a dead, inert computational node.

2.4 Smooth and Non-Saturating Variants: Leaky ReLU, ELU, SELU

To maintain backpropagation across the negative axis, several variants adjust the non-active regime:

Variant	Analytical Formula	Derivative
Leaky ReLU	$\max(\alpha z, z)$, with $\alpha \approx 0.01$	1 if $z > 0$ else α
PReLU	$\max(\alpha z, z)$, with learnable α	1 if $z > 0$ else α
ELU	z if $z > 0$ else $\alpha(e^z - 1)$	1 if $z > 0$ else αe^z
SELU	$\lambda \cdot \text{ELU}(z)$ $(\lambda \approx 1.0507,\ \alpha \approx 1.6733)$	λ if $z > 0$ else $\lambda\alpha e^z$

- **Leaky ReLU:** Fixes a small constant slope $\alpha = 0.01$ over the negative domain, ensuring $\frac{d\mathcal{L}}{dz} \neq 0$ and preventing units from permanently dying.
- **Parametric ReLU (PReLU):** Treats the negative slope α as a learnable parameter optimized directly through backpropagation.

- **Exponential Linear Unit (ELU):** Smooths the negative regime using an exponential curve, asymptotically saturating at $-\alpha$ to provide noise robustness while maintaining continuous differentiability across the transition point.
- **Scaled ELU (SELU):** Uses fixed values $\lambda \approx 1.0507$ and $\alpha \approx 1.6733$ derived via the Banach fixed-point theorem. When applied with specific weight initializations, it produces a *self-normalizing network* where activation variances naturally converge toward unit values across deep chains.

3 Weight Initialization: The Geometry of Variance Preservation

The initial state of parameter tensors determines whether activations and gradients explode or vanish during the initial forward and backward sweeps.

```
# The Failures of Trivial Initialization:
W_zeros = np.zeros((d_in, d_out))                # Breaks symmetry: all neurons update identically
W_const = np.ones((d_in, d_out))                 # Collapses downstream layer to rank 1
W_huge  = np.random.randn(d_in, d_out) * 10.0    # Extreme saturation -> vanishing gradients
W_tiny  = np.random.randn(d_in, d_out) * 0.0001 # Signals decay rapidly toward 0
```

Listing 1: The Failures of Trivial Initialization

3.1 The Symmetry Breaking Constraint

If every parameter in layer l is initialized to identical values ($w_{ij} = c$), then for any input vector $\mathbf{x}$, every hidden neuron computes the identical pre-activation z_j, emitting identical activations a_j. During backpropagation, the incoming gradients δ_j are identical, forcing identical parameter updates:

$$\Delta w_{1j} = \Delta w_{2j} = \cdots = \Delta w_{nj} \tag{10}$$

The entire layer acts as a single scalar unit, failing to learn diverse representations regardless of its width. Initializations must break symmetry randomly.

3.2 Variance Analysis of a Linear Layer

Consider an affine node taking n_{in} inputs: $z = \sum_{i=1}^{n_{\text{in}}} w_i x_i$ (omitting bias for clarity). Assume input features x_i and weights w_i are independent, identically distributed zero-mean random variables:

$$\mathbb{E}[x_i] = 0, \qquad \mathbb{E}[w_i] = 0 \tag{11}$$

The variance of the linear combination evaluates to:

$$\operatorname{Var}(z) = \operatorname{Var}\left(\sum_{i=1}^{n_{\text{in}}} w_i x_i\right) = \sum_{i=1}^{n_{\text{in}}} \operatorname{Var}(w_i x_i) \tag{12}$$

Using the independence property $\operatorname{Var}(AB) = \mathbb{E}[A]^2 \operatorname{Var}(B) + \mathbb{E}[B]^2 \operatorname{Var}(A) + \operatorname{Var}(A)\operatorname{Var}(B)$:

$$\operatorname{Var}(w_i x_i) = 0 \cdot \operatorname{Var}(x_i) + 0 \cdot \operatorname{Var}(w_i) + \operatorname{Var}(w_i)\operatorname{Var}(x_i) = \operatorname{Var}(w)\operatorname{Var}(x) \tag{13}$$

Summing across all n_{in} incoming paths:

$$\text{Var}(z) = n_{\text{in}} \, \text{Var}(w) \, \text{Var}(x) \tag{14}$$

To maintain a stable forward signal across layers without explosion or collapse, we require $\text{Var}(z) = \text{Var}(x)$. This sets our target parameter variance:

$$\boxed{\text{Var}(w) = \frac{1}{n_{\text{in}}}} \tag{15}$$

Conducting the exact symmetric derivation for the backward pass (where gradients accumulate across n_{out} outgoing connections) yields:

$$\text{Var}(w) = \frac{1}{n_{\text{out}}} \tag{16}$$

3.3 Xavier (Glorot) Initialization

Xavier Glorot and Yoshua Bengio (2010) unified the forward and backward constraints using their harmonic mean:

$$\text{Var}(w) = \frac{2}{n_{\text{in}} + n_{\text{out}}} \tag{17}$$

- **Xavier Normal:** Sample weights from $\mathcal{N}\left(0, \ \sigma^2 = \frac{2}{n_{\text{in}}+n_{\text{out}}}\right)$
- **Xavier Uniform:** A uniform distribution $\mathcal{U}(-a, a)$ with variance $\frac{a^2}{3} = \frac{2}{n_{\text{in}}+n_{\text{out}}}$ requires boundary limits:

$$a = \sqrt{\frac{6}{n_{\text{in}} + n_{\text{out}}}} \tag{18}$$

This formulation assumes symmetric activations with unit derivative around zero, making it suitable for tanh and linear units.

3.4 He (Kaiming) Initialization

Kaiming He et al. (2015) refined this derivation for ReLU non-linearities. Because ReLU sets all negative pre-activations to zero, it zeroes out roughly half of the incoming distribution volume:

$$\mathbb{E}[\text{ReLU}(z)^2] = \frac{1}{2} \text{Var}(z) \tag{19}$$

To account for this signal loss and preserve variance through layers, the forward variance must be doubled:

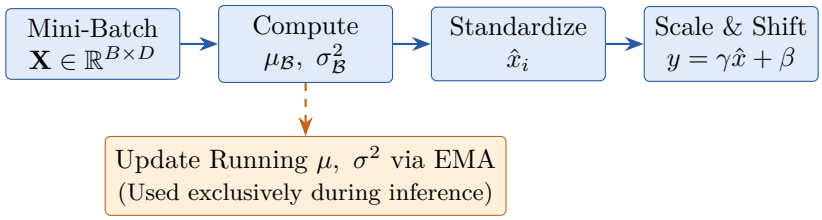
$$\boxed{\text{Var}(w) = \frac{2}{n_{\text{in}}}} \tag{20}$$

- **He Normal:** Sample from $\mathcal{N}\left(0, \ \sigma^2 = \frac{2}{n_{\text{in}}}\right)$
- **He Uniform:** Sample from $\mathcal{U}\left(-\sqrt{\frac{6}{n_{\text{in}}}}, \ \sqrt{\frac{6}{n_{\text{in}}}}\right)$

4 Batch Normalization: Mechanics and Exact Gradients

Sergey Ioffe and Christian Szegedy (2015) introduced Batch Normalization to explicitly standardize intermediate feature distributions during training.

Batch Normalization Pipeline



Given a mini-batch $\mathcal{B} = \{x_{1\ldots B}\}$ of dimension D (evaluated independently per feature dimension):

1. **Mini-batch Mean:**
$$\mu_{\mathcal{B}} = \frac{1}{B}\sum_{i=1}^{B} x_i \tag{21}$$

2. **Mini-batch Variance:**
$$\sigma^2_{\mathcal{B}} = \frac{1}{B}\sum_{i=1}^{B}(x_i - \mu_{\mathcal{B}})^2 \tag{22}$$

3. **Standardization:**
$$\hat{x}_i = \frac{x_i - \mu_{\mathcal{B}}}{\sqrt{\sigma^2_{\mathcal{B}} + \epsilon}} \tag{23}$$

4. **Scale and Shift:**
$$y_i = \gamma\hat{x}_i + \beta \tag{24}$$

Here, γ and β are learnable affine parameters that preserve representation capacity; if the network benefits from an unnormalized distribution, it can set $\gamma = \sqrt{\sigma^2}$ and $\beta = \mu$ to restore the raw pre-activations.

4.1 Exact Analytical Backpropagation through Batch Normalization

Batch Normalization links all examples in a mini-batch through the shared statistics $\mu_{\mathcal{B}}$ and $\sigma^2_{\mathcal{B}}$. The chain rule must take these cross-sample dependencies into account.

Let $\frac{\partial \mathcal{L}}{\partial y_i}$ be the incoming gradient from downstream layers, and let $\sigma_\epsilon = \sqrt{\sigma^2_{\mathcal{B}} + \epsilon}$.

The gradients for the learnable scale and shift parameters are:

$$\frac{\partial \mathcal{L}}{\partial \gamma} = \sum_{i=1}^{B} \frac{\partial \mathcal{L}}{\partial y_i}\hat{x}_i, \qquad \frac{\partial \mathcal{L}}{\partial \beta} = \sum_{i=1}^{B} \frac{\partial \mathcal{L}}{\partial y_i} \tag{25}$$

The gradient with respect to the standardized inputs $\hat{x}_i$ is:

$$\frac{\partial \mathcal{L}}{\partial \hat{x}_i} = \frac{\partial \mathcal{L}}{\partial y_i} \cdot \gamma \tag{26}$$

Next, we accumulate gradients into the batch variance:

$$\frac{\partial \mathcal{L}}{\partial \sigma^2_{\mathcal{B}}} = \sum_{i=1}^{B} \frac{\partial \mathcal{L}}{\partial \hat{x}_i}(x_i - \mu_{\mathcal{B}}) \cdot \left(-\frac{1}{2}(\sigma^2_{\mathcal{B}} + \epsilon)^{-3/2}\right) = -\frac{1}{2\sigma^3_\epsilon}\sum_{i=1}^{B} \frac{\partial \mathcal{L}}{\partial \hat{x}_i}(x_i - \mu_{\mathcal{B}}) \tag{27}$$

Now accumulate into the batch mean, combining the direct path and the path through the variance:

$$\frac{\partial \mathcal{L}}{\partial \mu_{\mathcal{B}}} = \left(\sum_{i=1}^{B} \frac{\partial \mathcal{L}}{\partial \hat{x}_i} \cdot \left(-\frac{1}{\sigma_\epsilon}\right)\right) + \frac{\partial \mathcal{L}}{\partial \sigma^2_{\mathcal{B}}} \cdot \frac{1}{B}\sum_{i=1}^{B} -2(x_i - \mu_{\mathcal{B}}) \tag{28}$$

Since $\sum_{i=1}^{B}(x_i - \mu_{\mathcal{B}}) = 0$, the second term drops out, leaving:

$$\frac{\partial \mathcal{L}}{\partial \mu_{\mathcal{B}}} = -\frac{1}{\sigma_\epsilon} \sum_{i=1}^{B} \frac{\partial \mathcal{L}}{\partial \hat{x}_i} \tag{29}$$

Finally, collecting all gradient paths back to the raw input x_i:

$$\frac{\partial \mathcal{L}}{\partial x_i} = \frac{\partial \mathcal{L}}{\partial \hat{x}_i} \cdot \frac{1}{\sigma_\epsilon} + \frac{\partial \mathcal{L}}{\partial \sigma_{\mathcal{B}}^2} \cdot \frac{2(x_i - \mu_{\mathcal{B}})}{B} + \frac{\partial \mathcal{L}}{\partial \mu_{\mathcal{B}}} \cdot \frac{1}{B} \tag{30}$$

Substituting the intermediate terms yields the compact, vectorized form:

$$\boxed{\frac{\partial \mathcal{L}}{\partial x_i} = \frac{\gamma}{B\sigma_\epsilon}\left[B\frac{\partial \mathcal{L}}{\partial y_i} - \frac{\partial \mathcal{L}}{\partial \beta} - \hat{x}_i \frac{\partial \mathcal{L}}{\partial \gamma}\right]} \tag{31}$$

```python
def batchnorm_backward(dout: np.ndarray, cache: tuple) -> tuple:
    """
    Exact analytical backward pass for Batch Normalization.
    dout: upstream gradient of shape (B, D)
    cache: (x_hat, gamma, var, eps, x_mu)
    """
    x_hat, gamma, var, eps, x_mu = cache
    B, D = dout.shape
    std_inv = 1.0 / np.sqrt(var + eps)

    dgamma = np.sum(dout * x_hat, axis=0)
    dbeta  = np.sum(dout, axis=0)

    # Gradient with respect to x_hat
    dxhat = dout * gamma

    # Vectorized gradient through mean and variance
    dx = (1.0 / B) * gamma * std_inv * (
        B * dxhat
        - np.sum(dxhat, axis=0)
        - x_hat * np.sum(dxhat * x_hat, axis=0)
    )

    return dx, dgamma, dbeta
```

Listing 2: Exact analytical backward pass for Batch Normalization

4.2 Training vs. Inference Discrepancy

At inference time, evaluate predictions on individual samples should not depend on batch statistics. During training, we maintain running estimates of the population statistics via Exponentially Weighted Moving Averages (EMA) with momentum factor α (typically 0.9 or 0.99):

$$\mu_{\text{run}} \leftarrow \alpha \mu_{\text{run}} + (1 - \alpha)\mu_{\mathcal{B}} \tag{32}$$

$$\sigma^2_{\text{run}} \leftarrow \alpha \sigma^2_{\text{run}} + (1 - \alpha)\sigma^2_{\mathcal{B}} \tag{33}$$

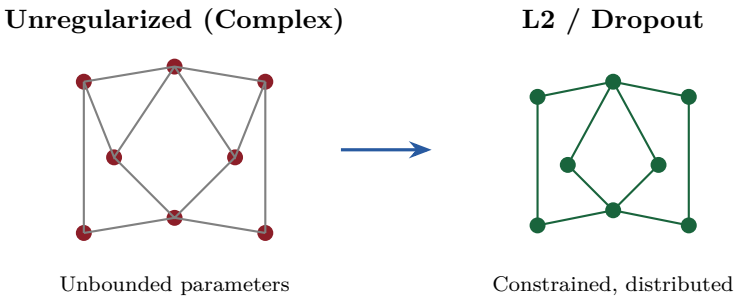
During inference, these precomputed scalar values replace the mini-batch statistics:

$$y = \gamma \left(\frac{x - \mu_{\text{run}}}{\sqrt{\sigma^2_{\text{run}} + \epsilon}} \right) + \beta \tag{34}$$

5 Regularization Techniques: Structural Constraints

Regularization modifies the learning process to prevent complex parameter configurations from fitting noise in the training set.

Regularization Geometry



5.1 Weight Decay (L_2 Regularization)

L_2 regularization augments the objective function with a penalty proportional to the squared Euclidean norm of the weight matrices:

$$\mathcal{L}_{\text{total}}(\mathbf{w}) = \mathcal{L}_0(\mathbf{w}) + \frac{\lambda}{2}\|\mathbf{w}\|_2^2 \tag{35}$$

Taking the gradient with respect to $\mathbf{w}$:

$$\nabla_{\mathbf{w}}\mathcal{L}_{\text{total}} = \nabla_{\mathbf{w}}\mathcal{L}_0 + \lambda\mathbf{w} \tag{36}$$

Under vanilla gradient descent, the parameter update becomes:

$$\mathbf{w}_{t+1} = \mathbf{w}_t - \eta(\nabla_{\mathbf{w}}\mathcal{L}_0 + \lambda\mathbf{w}_t) = (1-\eta\lambda)\mathbf{w}_t - \eta\nabla_{\mathbf{w}}\mathcal{L}_0 \tag{37}$$

At each step, the weights are shrunken by a decay factor $(1-\eta\lambda) < 1$ prior to receiving the gradient update. This suppresses large coefficients and drives unnecessary parameters toward zero.

5.2 Dropout: Ensemble Approximations via Stochastically Pruned Subgraphs

Nitish Srivastava, Geoffrey Hinton, et al. (2014) introduced Dropout, which stochastically deactivates a fraction of neurons at each forward pass.

```
# Inverted Dropout during training (p = keep_prob)
mask = (np.random.rand(*a.shape) < p) / p  # Inverted scaling
a_dropped = a * mask
```

Listing 3: Inverted Dropout during training

By scaling by $\frac{1}{p}$ during the forward pass (**Inverted Dropout**), activations maintain their expected magnitude, removing the need for any scaling adjustments at inference time:

$$\mathbb{E}[a_{\text{dropped}}] = \mathbb{E}\left[\frac{a \cdot \text{Bernoulli}(p)}{p}\right] = \frac{a \cdot p}{p} = a \tag{38}$$

Dropout provides two key regularizing benefits:

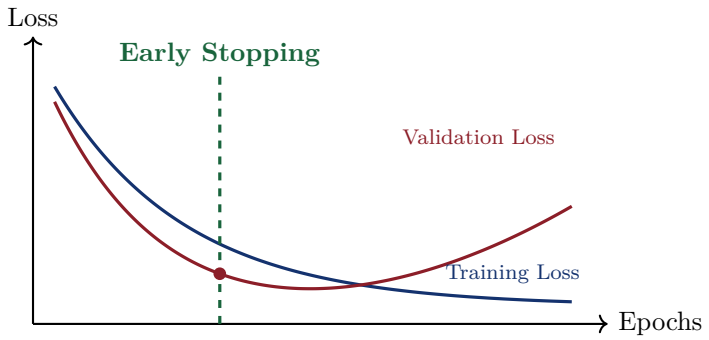
1. **Co-adaptation Destruction:** A neuron cannot rely on the guaranteed presence of specific upstream features, forcing each unit to learn useful, robust representations independently.

2. **Implicit Ensemble:** In a network of N units, dropping neurons samples from 2^N possible subnetworks. Training with dropout approximates bagging across these architectures using shared weights.

5.3 Early Stopping

Early stopping tracks performance on a held-out validation set and halts training once the validation loss stops improving for a specified number of checks (**patience**).

Early Stopping Criterion



This simple strategy bounds the optimization trajectory within an effective neighborhood around the initial parameters, preventing the model from fitting training noise.

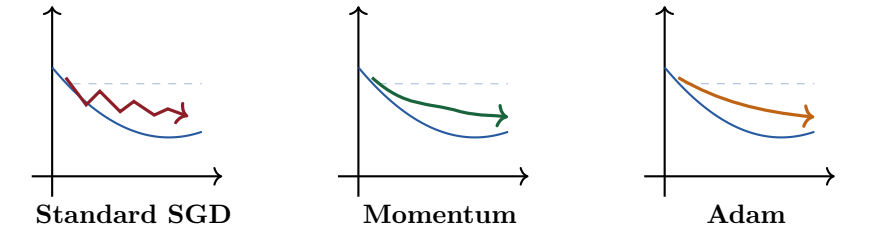
6 The Optimization Frontier: From Momentum to Adam

Standard gradient descent updates parameters along the negative direction of the loss gradient:

$$\mathbf{w}_{t+1} = \mathbf{w}_t - \eta \mathbf{g}_t, \qquad \mathbf{g}_t = \nabla_{\mathbf{w}} \mathcal{L}(\mathbf{w}_t) \tag{39}$$

In non-convex settings with anisotropic curvature, this leads to slow, oscillatory training trajectories.

Optimizer Trajectories in an Anisotropic Valley



6.1 The Exponentially Weighted Moving Average (EMA)

Modern optimizers rely on the Exponentially Weighted Moving Average (EMA) to smooth out noisy, short-term gradient signals:

$$v_t = \beta v_{t-1} + (1 - \beta)\theta_t \tag{40}$$

Unrolling the recursive formulation down to $t = 0$:

$$v_t = (1 - \beta) \sum_{i=1}^{t} \beta^{t-i} \theta_i \tag{41}$$

Because $\sum_{i=0}^{\infty} \beta^i = \frac{1}{1-\beta}$, the EMA acts as an average over the past approximately $\frac{1}{1-\beta}$ time steps.

6.2 Polyak Momentum

Polyak Momentum models the parameter vector as a physical particle with mass moving through a viscous fluid:

$$\mathbf{v}_t = \beta \mathbf{v}_{t-1} + \eta \mathbf{g}_t \tag{42}$$

$$\mathbf{w}_{t+1} = \mathbf{w}_t - \mathbf{v}_t \tag{43}$$

Gradients that oscillate back and forth cancel out within the accumulated velocity $\mathbf{v}_t$, while gradients that point in consistent directions compound. This dampens lateral oscillations and accelerates travel along valley floors.

6.3 Nesterov Accelerated Gradient (NAG)

Standard momentum applies the current gradient step and the accumulated momentum step simultaneously. Nesterov Accelerated Gradient computes the gradient after taking the momentum step, looking ahead to anticipate the next position:

$$\mathbf{v}_t = \beta \mathbf{v}_{t-1} + \eta \nabla_{\mathbf{w}} \mathcal{L}(\mathbf{w}_t - \beta \mathbf{v}_{t-1}) \tag{44}$$

$$\mathbf{w}_{t+1} = \mathbf{w}_t - \mathbf{v}_t \tag{45}$$

This predictive gradient helps slow down updates when heading up an opposing slope, reducing overshooting around sharp bends.

6.4 AdaGrad (Adaptive Gradient Algorithm)

AdaGrad tracks the cumulative sum of squared gradients along each coordinate direction:

$$s_t = s_{t-1} + \mathbf{g}_t \odot \mathbf{g}_t \tag{46}$$

$$\mathbf{w}_{t+1} = \mathbf{w}_t - \frac{\eta}{\sqrt{s_t + \epsilon}} \odot \mathbf{g}_t \tag{47}$$

Features that receive frequent, large gradients see their effective learning rates scaled down, while features with sparse, small gradients receive relatively larger steps.

However, because s_t accumulates squared gradients monotonically ($s_t \geq s_{t-1}$), the effective learning rate decays continuously, causing updates to stall prematurely in deep architectures.

6.5 RMSprop

Geoffrey Hinton introduced RMSprop to fix AdaGrad's vanishing learning rate problem by replacing the running sum with an exponentially decaying average:

$$s_t = \beta s_{t-1} + (1-\beta) \mathbf{g}_t \odot \mathbf{g}_t \tag{48}$$

$$\mathbf{w}_{t+1} = \mathbf{w}_t - \frac{\eta}{\sqrt{s_t + \epsilon}} \odot \mathbf{g}_t \tag{49}$$

This restricts the effective memory to recent time steps, scaling updates by local curvature without freezing parameter updates.

6.6 Adam (Adaptive Moment Estimation)

Kingma and Ba (2014) combined first-moment smoothing (momentum) and second-moment scaling (RMSprop) into Adam:

1. **First Moment (Mean of Gradients):**

$$_t = \beta_1 _{t-1} + (1-\beta_1)\mathbf{g}_t \tag{50}$$

2. **Second Moment (Uncentered Variance of Gradients):**

$$\mathbf{v}_t = \beta_2 \mathbf{v}_{t-1} + (1-\beta_2)(\mathbf{g}_t \odot \mathbf{g}_t) \tag{51}$$

6.6.1 Analytical Derivation of the Bias Correction Factor

Because moment vectors are typically initialized to zero ($_0 = \mathbf{0},\ \mathbf{v}_0 = \mathbf{0}$), both estimates remain biased toward zero during the opening iterations. Unrolling the first-moment recursion:

$$_t = (1-\beta_1)\sum_{i=1}^{t} \beta_1^{t-i}\mathbf{g}_i \tag{52}$$

Taking expectations on both sides under the assumption that the true gradient distribution has a locally stationary mean $\mathbb{E}[\mathbf{g}_i] \approx \mathbb{E}[\mathbf{g}_t]$:

$$\mathbb{E}[_t] = \mathbb{E}\left[(1-\beta_1)\sum_{i=1}^{t}\beta_1^{t-i}\mathbf{g}_i\right] = \mathbb{E}[\mathbf{g}_t](1-\beta_1)\sum_{i=1}^{t}\beta_1^{t-i} \tag{53}$$

Evaluating the finite geometric series:

$$\sum_{i=1}^{t}\beta_1^{t-i} = \sum_{k=0}^{t-1}\beta_1^k = \frac{1-\beta_1^t}{1-\beta_1} \tag{54}$$

Substituting this back gives:

$$\mathbb{E}[_t] = \mathbb{E}[\mathbf{g}_t](1-\beta_1)\left(\frac{1-\beta_1^t}{1-\beta_1}\right) = \mathbb{E}[\mathbf{g}_t](1-\beta_1^t) \tag{55}$$

To obtain an unbiased estimator $\mathbb{E}[\hat{}_t] = \mathbb{E}[\mathbf{g}_t]$, we must divide by $(1-\beta_1^t)$:

$$\hat{}_t = \frac{_t}{1-\beta_1^t} \tag{56}$$

The same derivation applies to the second-moment estimator:

$$\hat{\mathbf{v}}_t = \frac{\mathbf{v}_t}{1-\beta_2^t} \tag{57}$$

Using these bias-corrected estimates, the Adam update step evaluates to:

$$\boxed{\mathbf{w}_{t+1} = \mathbf{w}_t - \frac{\eta}{\sqrt{\hat{\mathbf{v}}_t}+\epsilon}\odot\hat{}_t} \tag{58}$$

7 Building a Production-Grade Deep Linear Block

Below, we construct an integrated neural layer containing affine weights, Batch Normalization, and a modern activation function, accompanied by clean forward and backward implementations.

```
import numpy as np

class LinearBatchNormRelu:
    """
    A unified, production-grade deep learning block:
    Affine -> Batch Normalization -> Leaky ReLU.
    Implements exact numerical forward and backward passes.
    """
    def __init__(self, in_features: int, out_features: int,
                 alpha: float = 0.01, eps: float = 1e-5):
        # He (Kaiming) Normal Initialization for non-saturating activations
        self.W = np.random.randn(in_features, out_features) * np.sqrt(2.0 / in_features)
        self.b = np.zeros((1, out_features))

        # Batch Normalization learnable parameters
        self.gamma = np.ones((1, out_features))
        self.beta  = np.zeros((1, out_features))

        # Running statistics for inference
        self.running_mean = np.zeros((1, out_features))
        self.running_var  = np.ones((1, out_features))

        self.alpha = alpha  # Leaky ReLU slope
        self.eps = eps
        self.cache = {}

    def forward(self, X: np.ndarray, training: bool = True) -> np.ndarray:
        # 1. Affine Projection
        Z = np.dot(X, self.W) + self.b
        B, D = Z.shape

        # 2. Batch Normalization
        if training:
            mean = np.mean(Z, axis=0, keepdims=True)
            var  = np.var(Z, axis=0, keepdims=True)

            # Standardization
            Z_hat = (Z - mean) / np.sqrt(var + self.eps)
            BN_out = self.gamma * Z_hat + self.beta

            # Update running statistics via EMA
            momentum = 0.9
            self.running_mean = momentum * self.running_mean + (1.0 - momentum) * mean
            self.running_var  = momentum * self.running_var  + (1.0 - momentum) * var

            self.cache["bn"] = (Z, Z_hat, mean, var, X)
        else:
```

```
            # Inference pass using precomputed running statistics
            Z_hat = (Z - self.running_mean) / np.sqrt(self.running_var
                + self.eps)
            BN_out = self.gamma * Z_hat + self.beta
            self.cache["bn"] = (Z, Z_hat, self.running_mean, self.
                running_var, X)

        # 3. Leaky ReLU Activation
        A = np.where(BN_out > 0, BN_out, self.alpha * BN_out)
        self.cache["act"] = (BN_out, A)
        return A

    def backward(self, dOut: np.ndarray) -> tuple:
        """
        Exact backward pass via the multi-variable chain rule.
        dOut: upstream gradient of shape (B, out_features).
        """
        BN_out, A = self.cache["act"]
        Z, Z_hat, mean, var, X = self.cache["bn"]
        B, D = dOut.shape

        # 1. Backprop through Leaky ReLU
        dBN = np.where(BN_out > 0, dOut, self.alpha * dOut)

        # 2. Backprop through Batch Normalization
        dgamma = np.sum(dBN * Z_hat, axis=0, keepdims=True)
        dbeta  = np.sum(dBN, axis=0, keepdims=True)

        std_inv = 1.0 / np.sqrt(var + self.eps)
        dZ_hat = dBN * self.gamma

        # Exact vectorized gradient through mini-batch statistics
        dZ = (1.0 / B) * self.gamma * std_inv * (
            B * dZ_hat
            - np.sum(dZ_hat, axis=0, keepdims=True)
            - Z_hat * np.sum(dZ_hat * Z_hat, axis=0, keepdims=True)
        )

        # 3. Backprop through Affine Transformation
        dW = np.dot(X.T, dZ)
        db = np.sum(dZ, axis=0, keepdims=True)
        dX = np.dot(dZ, self.W.T)

        return dX, dW, db, dgamma, dbeta
```

Listing 4: Unified Affine + BatchNorm + LeakyReLU block with exact backward pass

```
class AdamOptimizer:
    """
    Vectorized Adam Optimizer with decoupled bias correction tracking.
    """
    def __init__(self, lr: float = 0.001, beta1: float = 0.9,
                 beta2: float = 0.999, eps: float = 1e-8):
        self.lr = lr
        self.beta1 = beta1
        self.beta2 = beta2
        self.eps = eps
        self.m = {}
```

```
        self.v = {}
        self.t = 0

    def step(self, param: np.ndarray, grad: np.ndarray, param_id: str)
        -> np.ndarray:
        if param_id not in self.m:
            self.m[param_id] = np.zeros_like(param)
            self.v[param_id] = np.zeros_like(param)

        self.t += 1

        # Update biased first and second moments
        self.m[param_id] = self.beta1 * self.m[param_id] + (1.0 - self.
            beta1) * grad
        self.v[param_id] = self.beta2 * self.v[param_id] + (1.0 - self.
            beta2) * (grad ** 2)

        # Bias corrections
        m_hat = self.m[param_id] / (1.0 - self.beta1 ** self.t)
        v_hat = self.v[param_id] / (1.0 - self.beta2 ** self.t)

        # Apply update step
        param -= self.lr * m_hat / (np.sqrt(v_hat) + self.eps)
        return param
```

Listing 5: Vectorized Adam Optimizer with decoupled bias correction tracking

8 Diagnostic Guide

Symptom	Underlying Cause	Intervention
Loss evaluates to NaN / Inf	Exploding gradients; Pre-activations overflowing	Switch from standard initialization to He/Xavier; Enforce gradient clipping: $\mathbf{g} \leftarrow \mathbf{g} \cdot \min(1, c/\|\mathbf{g}\|_2)$; Lower learning rate by an order of magnitude.
Dead units / Flat loss	Dying ReLU problem; Saturating output layers	Replace ReLU with Leaky ReLU or ELU; Initialize biases to small positive constants (e.g., 0.01); Lower learning rate.
Validation loss diverges while training loss drops	Model capacity fitting noise; Unconstrained weight magnitudes	Introduce Dropout layers ($p = 0.2$ to 0.5); Apply L2 weight decay; Add early stopping callback with patience checks.
Severe optimization oscillations	High anisotropic curvature; Inappropriate learning rate	Normalize input features (StandardScaler); Add Batch Normalization layers; Switch optimizer to Adam or SGD with Momentum.

Exercises

1. **Analytical Variance of Leaky ReLU:** Let $z \sim \mathcal{N}(0, \sigma^2)$, and let $a = \max(\alpha z, z)$. Prove analytically that:

$$\mathbb{E}[a^2] = \frac{1 + \alpha^2}{2}\sigma^2 \tag{59}$$

Use this result to derive the corresponding He-style weight initialization variance for Leaky ReLU networks.

2. **Invariance to Weight Scaling in Batch Normalization:** Let $\mathbf{W}' = \kappa\mathbf{W}$, where $\kappa > 0$ is an arbitrary positive scalar. Prove that the output of an affine transformation followed by Batch Normalization is invariant to κ:

$$\mathrm{BN}(\mathbf{W}'\mathbf{x}) = \mathrm{BN}(\mathbf{W}\mathbf{x}) \tag{60}$$

Next, show that the parameter gradients satisfy:

$$\nabla_{\mathbf{W}'}\mathcal{L} = \frac{1}{\kappa}\nabla_{\mathbf{W}}\mathcal{L} \tag{61}$$

Explain how this relationship stabilizes the optimization of layers followed by Batch Normalization.

3. **Equivalence of Weight Decay and L2 Regularization:** Show that optimizing an objective with L_2 regularization using standard gradient descent is mathematically equivalent to optimizing the unregularized objective using decoupled weight decay. Then, explain why this equivalence breaks down when using adaptive gradient algorithms like Adam (giving rise to **AdamW**).

Chapter 3

Spatial Inductive Biases

Convolutions, Pooling, and Visual Architectures

A Comprehensive Treatment of Convolutional Neural Networks

From Biological Foundations to Modern Architectures

Contents

1 The Breakdown of Dense Layers on Spatial Data **2**

2 Biological Foundations: The Architecture of the Visual Cortex **3**

3 The Forward Mechanics: Convolutions, Strides, and Padding **3**
3.1 The 2D Convolution Operator 4
3.2 Convolutions over Volumes (Multi-Channel Inputs) 4
3.3 Padding (P) 5
3.4 Strides (S) and Output Spatial Geometry 5
3.5 Parameter Counting: Dense vs. Convolutional 5

4 Spatial Pooling Mechanics and Invariance **6**
4.1 Pooling Variants 6
4.2 Properties of Pooling Layers 7

5 Backpropagation Through Convolutions, Pooling, and Flattening **7**
5.1 1. Backpropagation Through Flattening 7
5.2 2. Backpropagation Through Max Pooling 7
5.3 3. Backpropagation Through the Convolution Layer 8
5.3.1 A. Gradient with Respect to Bias (b) 8
5.3.2 B. Gradient with Respect to Weights ($\mathbf{W}$) 8
5.3.3 C. Gradient with Respect to Input ($\mathbf{X}$) 9

6 Building a Minimal CNN Engine From Scratch **9**

7 Classical and Modern Architectural Paradigms **12**
7.1 LeNet-5 (1998) 12
7.2 The VGG Philosophy: Small Kernels over Large Ones 13

8 Transfer Learning, Data Augmentation, and Practical Workflows **13**
8.1 Transfer Learning Protocols 13
8.2 Data Augmentation 14

9 Diagnostic Guide **15**

Exercises **15**

1 The Breakdown of Dense Layers on Spatial Data

In Chapters 1 and 2, our networks treated input vectors as arbitrary, unstructured lists of numbers. Every unit in layer l was fully connected to every unit in layer $l-1$. If you permuted the order of input features identically across all samples, a fully connected network would achieve the exact same loss curve.

When processing images, that spatial agnosticism is a catastrophe.

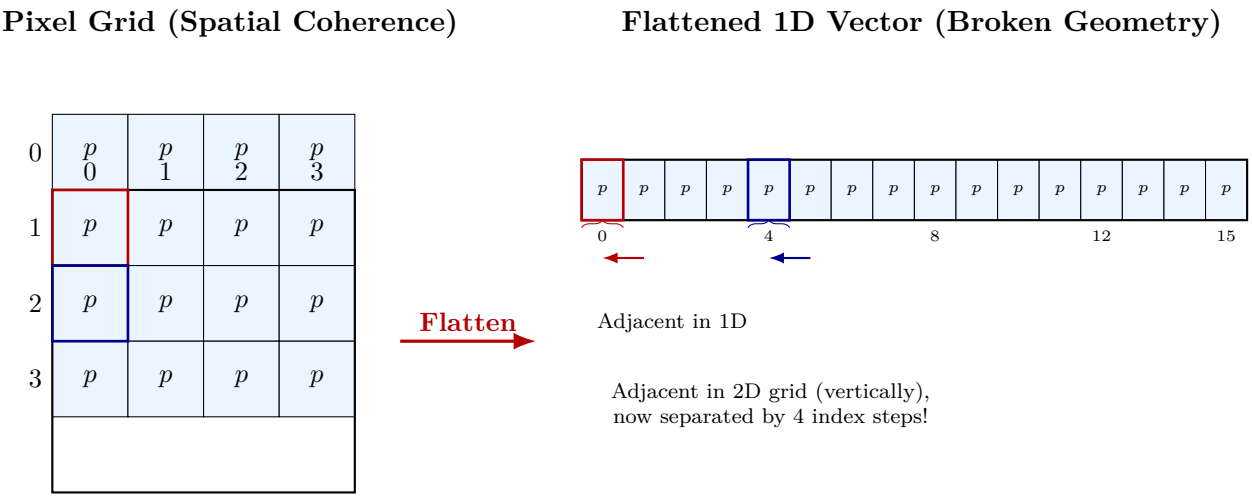


Figure 1: Flattening a 2D pixel grid destroys spatial adjacency. The red cell (row 0, col 0) and blue cell (row 1, col 0) are vertically adjacent in the original grid but end up at indices 0 and 4 in the flattened vector.

Consider three failure modes that arise when applying standard Multi-Layer Perceptrons (MLPs) directly to images:

1. **Parameter Explosion and Computational Cost:** Suppose we take a modest color image of dimensions $224 \times 224 \times 3$. Flattening this grid yields an input vector $\mathbf{x} \in \mathbb{R}^{150{,}528}$. If the very first hidden dense layer contains just 1,024 neurons, that single affine matrix requires:

$$\text{Parameters} = 150{,}528 \times 1{,}024 \approx 1.54 \times 10^8 \text{ weights} \tag{1}$$

   Allocating hundreds of millions of parameters to a single layer consumes gigabytes of memory, throttles training throughput, and makes deep stacks computationally intractable.

2. **Severe Overfitting:** A dense layer with 10^8 free parameters has sufficient capacity to memorize arbitrary pixel configurations and noise rather than extracting generalizable visual representations.

3. **Loss of Spatial Topology and Translation Invariance:** In an image, physical semantics are encoded by local spatial neighborhoods. A pixel at coordinate (i, j) is strongly correlated with $(i+1, j)$ and $(i, j+1)$. Flattening a 2D matrix into a 1D column vector strips away vertical adjacency: two vertically adjacent pixels suddenly find themselves separated by thousands of indices. Furthermore, if an object shifts two pixels to the right, a dense network perceives an entirely different set of input features and must re-learn the object's features from scratch across different weight parameters.

To process images efficiently, we introduce two structural inductive biases directly into the network architecture:

- **Sparse Connectivity (Local Receptive Fields):** Neurons compute affine transformations over small, local neighborhoods of pixels rather than the entire image.
- **Weight Sharing:** The same parameter tensor (filter) is swept across the entire input space, guaranteeing that feature detection operates identically regardless of absolute spatial coordinates.

2 Biological Foundations: The Architecture of the Visual Cortex

The design of the Convolutional Neural Network (CNN) is directly grounded in experimental neurophysiology.

In 1959 and 1962, David Hubel and Torsten Wiesel conducted a landmark series of experiments at Johns Hopkins University on the primary visual cortex (area V1) of anesthetized cats. By inserting microelectrodes into individual cortical neurons while projecting visual stimuli onto a screen, they observed that neurons in V1 did not fire in response to diffuse light or whole-scene illumination. Instead, individual cells fired selectively when lines, bars, or edges of a specific orientation were swept across a confined sub-region of the visual field.

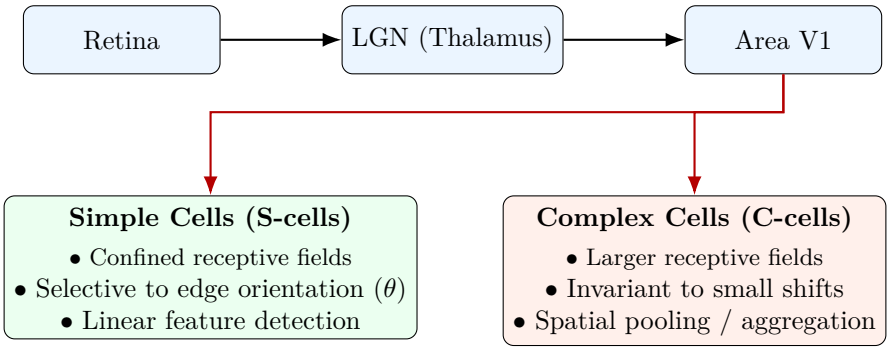


Figure 2: Hierarchical visual pathway from retina through LGN to primary visual cortex (V1), showing the two functional cell types discovered by Hubel and Wiesel.

Hubel and Wiesel discovered two distinct functional cell types in the visual pathway:

- **Simple Cells:** Exhibit small, localized receptive fields. They activate strongly only when an edge of a specific orientation (e.g., a vertical bar) appears at a precise coordinate.
- **Complex Cells:** Receive convergent projections from groups of simple cells. They maintain orientation selectivity over larger receptive fields while tolerating small spatial translations. The edge can shift slightly within the receptive window without reducing the cell's firing rate.

In 1980, Kunihiko Fukushima formalized this biological hierarchy into a computational model called the **Neocognitron**, alternating between S-cells (feature extraction) and C-cells (spatial downsampling). In 1989 and 1998, Yann LeCun et al. unified Fukushima's topological hierarchy with end-to-end gradient-based backpropagation, creating **LeNet-5**—the foundational blueprint for modern computer vision architectures.

3 The Forward Mechanics: Convolutions, Strides, and Padding

In deep learning literature, the core linear operator of a CNN is termed a convolution, though it is technically implemented as cross-correlation (the kernel is not flipped prior to sliding).

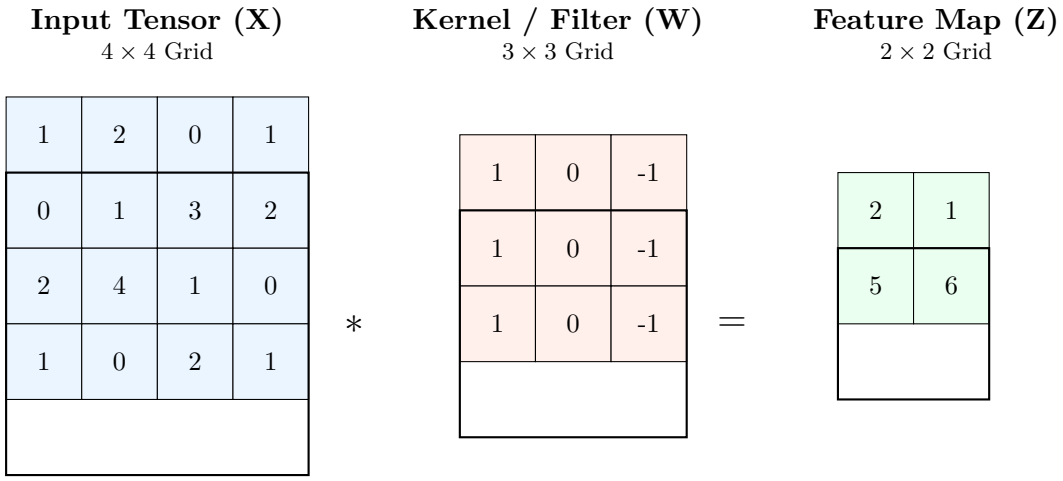


Figure 3: 2D convolution of a 4×4 input with a 3×3 vertical-edge kernel, producing a 2×2 feature map.

3.1 The 2D Convolution Operator

Given a 2D single-channel input $\mathbf{X} \in \mathbb{R}^{H \times W}$ and a 2D filter $\mathbf{W} \in \mathbb{R}^{K \times K}$ with scalar bias b, the pre-activation value at output coordinate (i, j) is computed via the Frobenius inner product of the kernel with the matching input patch:

$$Z_{i,j} = (\mathbf{X} * \mathbf{W})_{i,j} + b = \sum_{u=0}^{K-1} \sum_{v=0}^{K-1} X_{i+u,j+v} W_{u,v} + b \tag{2}$$

An oriented filter detects contrast shifts by applying opposing sign weights across spatial axes. For instance, a horizontal Sobel-style kernel subtracts bottom pixels from top pixels to detect horizontal boundaries:

$$\mathbf{W}_{\text{horizontal}} = \begin{bmatrix} -1 & -1 & -1 \\ 0 & 0 & 0 \\ 1 & 1 & 1 \end{bmatrix}, \quad \mathbf{W}_{\text{vertical}} = \begin{bmatrix} -1 & 0 & 1 \\ -1 & 0 & 1 \\ -1 & 0 & 1 \end{bmatrix} \tag{3}$$

In modern deep networks, we do not hand-engineer these coefficients. We initialize kernel weights randomly and let backpropagation discover optimal feature detectors automatically.

3.2 Convolutions over Volumes (Multi-Channel Inputs)

Images are three-dimensional arrays: $\mathbf{X} \in \mathbb{R}^{H \times W \times C_{\text{in}}}$, where $C_{\text{in}} = 3$ for RGB color data.

To convolve over a multi-channel volume, **a filter must match the full channel depth of its input**. A single filter is parameterized as a 3D tensor $\mathbf{W} \in \mathbb{R}^{K \times K \times C_{\text{in}}}$.

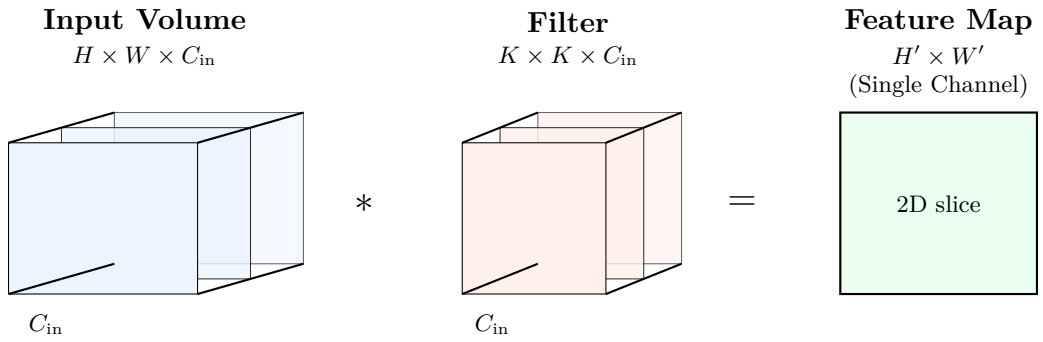


Figure 4: Convolution over a multi-channel input volume. Each filter matches the full channel depth C_{in} and produces one 2D feature map.

To compute an output coordinate, the filter computes the elementwise product across all spatial coordinates and channels simultaneously, collapsing them into a single scalar:

$$Z_{i,j} = \sum_{c=0}^{C_{\text{in}}-1} \sum_{u=0}^{K-1} \sum_{v=0}^{K-1} X_{i+u,j+v,c} W_{u,v,c} + b \tag{4}$$

One filter produces one 2D spatial feature map. If a convolutional layer uses C_{out} distinct filters, their individual 2D outputs stack together along the channel axis to create an output volume $\mathbf{Z} \in \mathbb{R}^{H' \times W' \times C_{\text{out}}}$.

3.3 Padding (P)

When sliding a $K \times K$ kernel across an unpadded $N \times N$ image, boundary pixels are sampled far fewer times than interior pixels, causing the spatial grid to shrink by $K-1$ pixels after every layer.

To preserve spatial dimensions and prevent edge information loss, we pad the borders of the input tensor with zeros (**zero-padding**):

- **Valid Padding ($P = 0$):** No padding is applied. The spatial dimension shrinks strictly.
- **Same Padding:** The input is padded with P zeros along every border so that the output spatial resolution matches the input resolution when using a unit stride ($S = 1$):

$$P = \frac{K-1}{2} \quad \text{(for odd kernel sizes } K\text{)} \tag{5}$$

3.4 Strides (S) and Output Spatial Geometry

The stride S dictates the step size the filter advances across spatial axes. Advancing by $S > 1$ downsamples the resulting feature map directly.

Given input spatial size N, kernel size K, padding P, and stride S, the output spatial dimension M is governed by the discrete floor equation:

$$M = \left\lfloor \frac{N + 2P - K}{S} \right\rfloor + 1 \tag{6}$$

3.5 Parameter Counting: Dense vs. Convolutional

The total parameter count of a convolutional layer depends strictly on its kernel geometry and channel depths—**completely independent of input height and width**:

$$\text{Parameters} = C_{\text{out}} \times (K \times K \times C_{\text{in}} + 1) \tag{7}$$

- **Example:** A layer with 64 filters of size 3×3 applied to an RGB input ($C_{\text{in}} = 3$) has:

$$\text{Weights} = 64 \times (3 \times 3 \times 3) = 1{,}728, \quad \text{Biases} = 64 \implies \text{Total} = 1{,}792 \tag{8}$$

- If the input image resolution scales from 32×32 to 4096×4096, the parameter count remains exactly 1,792. This parameter efficiency enables deep convolutional architectures to scale cleanly to high-resolution imagery.

4 Spatial Pooling Mechanics and Invariance

A convolutional layer is typically followed by an elementwise non-linear activation (e.g., ReLU), and then routed into a **Pooling Layer**.

Pooling layers downsample feature map activations to reduce memory usage and parameter counts while building spatial invariance.

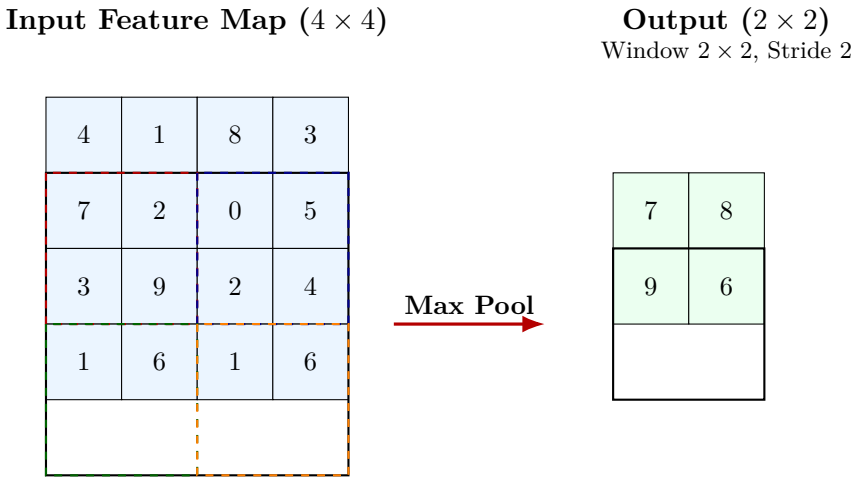


Figure 5: Max pooling with a 2×2 window and stride 2 reduces a 4×4 feature map to 2×2 by selecting the maximum value in each non-overlapping window.

4.1 Pooling Variants

1. **Max Pooling:** Slides a $P_h \times P_w$ window across each feature map independently and extracts the maximum scalar activation:

$$Y_{i,j} = \max_{u \in [0, P_h - 1],\ v \in [0, P_w - 1]} X_{i \cdot S + u,\ j \cdot S + v} \tag{9}$$

   Max pooling acts as a local feature selector: it reports whether a visual feature (such as an edge or texture) was detected anywhere within the window, regardless of its exact location.

2. **Average Pooling:** Computes the arithmetic mean over the pooling window:

$$Y_{i,j} = \frac{1}{P_h P_w} \sum_{u=0}^{P_h - 1} \sum_{v=0}^{P_w - 1} X_{i \cdot S + u,\ j \cdot S + v} \tag{10}$$

   Average pooling smooths activations uniformly and was standard in early models like LeNet-5, but modern networks predominantly favor Max Pooling or Strided Convolutions for feature extraction.

3. **Global Average Pooling (GAP):** Takes a full feature map of size $H \times W \times C$ and collapses each 2D channel slice into a single average value, outputting a $1 \times 1 \times C$ vector:

$$\text{GAP}(X)_c = \frac{1}{H \cdot W} \sum_{i=1}^{H} \sum_{j=1}^{W} X_{i,j,c} \tag{11}$$

Replacing massive flattening and dense layers with GAP drastically reduces parameter counts and acts as a structural regularizer against overfitting.

4.2 Properties of Pooling Layers

- **Zero Trainable Parameters:** Pooling operations have no learnable weights or biases. They are fixed, non-parametric aggregation operations.
- **Channel Preservation:** Pooling processes each channel slice independently. The input channel depth remains unchanged: $C_{\text{out}} = C_{\text{in}}$.
- **Local Translation Invariance:** If an input pattern shifts by a pixel or two, its maximum activation remains within the pooling window, yielding the exact same output value and stabilizing downstream classifications.

5 Backpropagation Through Convolutions, Pooling, and Flattening

Training a CNN requires propagating scalar loss gradients backward through convolutions, pooling operations, and tensor reshaping.

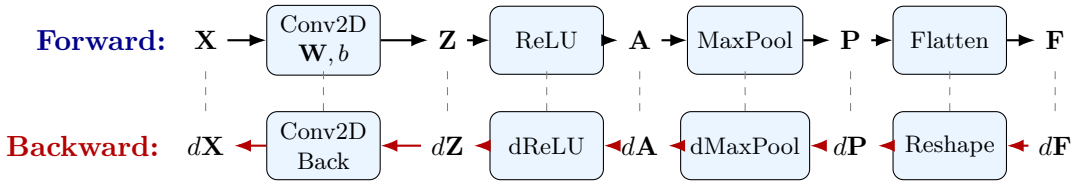


Figure 6: Forward and backward computational graph through a convolutional layer, ReLU, max pooling, and flattening.

5.1 1. Backpropagation Through Flattening

During forward propagation, a 3D feature tensor $\mathbf{P} \in \mathbb{R}^{H \times W \times C}$ is reshaped into a 1D feature vector $\mathbf{f} \in \mathbb{R}^{D}$ (where $D = H \cdot W \cdot C$):

$$\mathbf{f} = \text{vec}(\mathbf{P}) \tag{12}$$

Because reshaping is an indexing operation without parameter scaling, the upstream gradient vector $\frac{\partial L}{\partial \mathbf{f}} \in \mathbb{R}^{D}$ maps directly back to the original tensor geometry:

$$\frac{\partial L}{\partial \mathbf{P}} = \text{reshape}\left(\frac{\partial L}{\partial \mathbf{f}}, (H, W, C)\right) \tag{13}$$

5.2 2. Backpropagation Through Max Pooling

Max pooling selects the maximum value within each local pooling window and discards the rest.

During backpropagation, incoming gradients flow exclusively to the specific coordinate that produced the maximum value during the forward pass. All non-maximal coordinates receive a gradient of zero:

$$\frac{\partial L}{\partial X_{i,j}} = \sum_{(m,n)\in\Omega(i,j)} \frac{\partial L}{\partial Y_{m,n}} \cdot \mathbb{I}\left(X_{i,j} == \max_{(u,v)} X_{m\cdot S+u,\ n\cdot S+v}\right) \tag{14}$$

```python
def maxpool_backward(dY: np.ndarray, X: np.ndarray, pool_size: int = 2, stride: int = 2) -> np.ndarray:
    """
    Routes upstream gradient dY back to the input coordinates that held the maximum value.
    """
    H, W = X.shape
    dX = np.zeros_like(X)
    H_out, W_out = dY.shape

    for i in range(H_out):
        for j in range(W_out):
            h_start, h_end = i * stride, i * stride + pool_size
            w_start, w_end = j * stride, j * stride + pool_size

            patch = X[h_start:h_end, w_start:w_end]
            max_val = np.max(patch)

            # Create a boolean mask of where the maximum occurred
            mask = (patch == max_val)
            # Route gradient exclusively through the maximal coordinate
            dX[h_start:h_end, w_start:w_end] += mask * dY[i, j]

    return dX
```

Listing 1: Max pooling backward pass in NumPy.

5.3 3. Backpropagation Through the Convolution Layer

Let single-channel input $\mathbf{X}$ convolve with single-channel kernel $\mathbf{W}$ to produce pre-activation grid $\mathbf{Z} = \mathbf{X} * \mathbf{W} + b$. We receive the upstream error tensor $\delta = \frac{\partial L}{\partial \mathbf{Z}}$.

5.3.1 A. Gradient with Respect to Bias (b)

Because the scalar bias b is added to every entry $Z_{i,j}$, its gradient accumulates across all spatial coordinates:

$$\frac{\partial L}{\partial b} = \sum_i \sum_j \frac{\partial L}{\partial Z_{i,j}} = \sum_{i,j} \delta_{i,j} \tag{15}$$

5.3.2 B. Gradient with Respect to Weights (W)

Applying the multivariable chain rule:

$$\frac{\partial L}{\partial W_{u,v}} = \sum_i \sum_j \frac{\partial L}{\partial Z_{i,j}} \frac{\partial Z_{i,j}}{\partial W_{u,v}} \tag{16}$$

Since $Z_{i,j} = \sum_{u,v} X_{i+u,j+v} W_{u,v} + b$, we have $\frac{\partial Z_{i,j}}{\partial W_{u,v}} = X_{i+u,j+v}$. Therefore:

$$\frac{\partial L}{\partial W_{u,v}} = \sum_i \sum_j \delta_{i,j} X_{i+u,j+v} \tag{17}$$

The gradient with respect to the kernel $\mathbf{W}$ is computed by **convolving the input tensor $\mathbf{X}$ with the upstream gradient tensor δ.**

5.3.3 C. Gradient with Respect to Input (X)

To pass error signals back to earlier layers, we compute $\frac{\partial L}{\partial \mathbf{X}}$:

$$\frac{\partial L}{\partial X_{i,j}} = \sum_u \sum_v \frac{\partial L}{\partial Z_{i-u,j-v}} \frac{\partial Z_{i-u,j-v}}{\partial X_{i,j}} = \sum_u \sum_v \delta_{i-u,j-v} W_{u,v} \tag{18}$$

This equation is a full convolution between the zero-padded upstream gradient δ and the kernel $\mathbf{W}$ rotated by 180° ($\mathbf{W}^{\text{rot180}}$):

$$\frac{\partial L}{\partial \mathbf{X}} = \delta *_{\text{full}} \mathbf{W}^{\text{rot180}} \tag{19}$$

6 Building a Minimal CNN Engine From Scratch

Below is a complete, self-contained CNN implementation written in pure Python and NumPy. It implements explicit 2D convolution, ReLU, Max Pooling, and Dense output layers with exact forward and backward passes.

```
import numpy as np

class Conv2D:
    def __init__(self, in_channels: int, out_channels: int, kernel_size: int = 3):
        self.k = kernel_size
        self.in_c = in_channels
        self.out_c = out_channels

        # He (Kaiming) Normal Initialization
        limit = np.sqrt(2.0 / (in_channels * self.k * self.k))
        self.W = np.random.randn(out_channels, in_channels, self.k, self.k) * limit
        self.b = np.zeros((out_channels, 1))

    def forward(self, X: np.ndarray) -> np.ndarray:
        # X shape: (C_in, H, W)
        self.X = X
        C_in, H, W = X.shape
        H_out = H - self.k + 1
        W_out = W - self.k + 1

        self.Z = np.zeros((self.out_c, H_out, W_out))
        for c_out in range(self.out_c):
            for i in range(H_out):
                for j in range(W_out):
                    patch = X[:, i:i+self.k, j:j+self.k]
                    self.Z[c_out, i, j] = np.sum(patch * self.W[c_out]) + self.b[c_out]
        return self.Z

    def backward(self, dZ: np.ndarray, lr: float = 0.01) -> np.ndarray:
```

```
        # dZ shape: (C_out, H_out, W_out)
        C_in, H, W = self.X.shape
        H_out, W_out = dZ.shape[1], dZ.shape[2]

        dW = np.zeros_like(self.W)
        db = np.zeros_like(self.b)
        dX = np.zeros_like(self.X)

        for c_out in range(self.out_c):
            db[c_out] = np.sum(dZ[c_out])
            for i in range(H_out):
                for j in range(W_out):
                    patch = self.X[:, i:i+self.k, j:j+self.k]
                    dW[c_out] += patch * dZ[c_out, i, j]
                    dX[:, i:i+self.k, j:j+self.k] += self.W[c_out] * dZ
    [c_out, i, j]

        # SGD updates
        self.W -= lr * dW
        self.b -= lr * db
        return dX


class MaxPool2D:
    def __init__(self, size: int = 2, stride: int = 2):
        self.size = size
        self.stride = stride

    def forward(self, X: np.ndarray) -> np.ndarray:
        self.X = X
        C, H, W = X.shape
        H_out = (H - self.size) // self.stride + 1
        W_out = (W - self.size) // self.stride + 1

        out = np.zeros((C, H_out, W_out))
        for c in range(C):
            for i in range(H_out):
                for j in range(W_out):
                    h_start, w_start = i * self.stride, j * self.stride
                    patch = X[c, h_start:h_start+self.size, w_start:
    w_start+self.size]
                    out[c, i, j] = np.max(patch)
        return out

    def backward(self, dY: np.ndarray) -> np.ndarray:
        C, H, W = self.X.shape
        H_out, W_out = dY.shape[1], dY.shape[2]
        dX = np.zeros_like(self.X)

        for c in range(C):
            for i in range(H_out):
                for j in range(W_out):
                    h_start, w_start = i * self.stride, j * self.stride
                    patch = self.X[c, h_start:h_start+self.size,
    w_start:w_start+self.size]
                    mask = (patch == np.max(patch))
                    dX[c, h_start:h_start+self.size, w_start:w_start+
    self.size] += mask * dY[c, i, j]
```

```
        return dX


class ToyCNN:
    """
    A minimal end-to-end CNN classifier:
    Conv2D -> ReLU -> MaxPool2D -> Flatten -> Dense(1) -> Sigmoid
    """
    def __init__(self):
        self.conv = Conv2D(in_channels=1, out_channels=2, kernel_size=3)
        self.pool = MaxPool2D(size=2, stride=2)
        # Assuming input shape: (1, 8, 8)
        # Conv out: (2, 6, 6) -> Pool out: (2, 3, 3) -> Flatten: (18,)
        self.W_fc = np.random.randn(18, 1) * np.sqrt(2.0 / 18)
        self.b_fc = 0.0

    def forward(self, x: np.ndarray) -> float:
        self.z_conv = self.conv.forward(x)
        self.a_conv = np.maximum(0, self.z_conv) # ReLU
        self.pooled = self.pool.forward(self.a_conv)
        self.flat = self.pooled.reshape(-1, 1)   # Flatten to column vector

        self.z_fc = np.dot(self.W_fc.T, self.flat) + self.b_fc
        self.out = 1.0 / (1.0 + np.exp(-np.clip(self.z_fc[0, 0], -30, 30)))
        return self.out

    def train_step(self, x: np.ndarray, y: float, lr: float = 0.05) -> float:
        y_hat = self.forward(x)
        loss = -(y * np.log(y_hat + 1e-12) + (1.0 - y) * np.log(1.0 - y_hat + 1e-12))

        # Output delta
        delta_fc = y_hat - y
        dW_fc = self.flat * delta_fc
        db_fc = delta_fc

        # Propagate back to flattened features
        dflat = self.W_fc * delta_fc
        dpool = dflat.reshape(self.pooled.shape)

        # Backprop through pooling and ReLU
        dact = self.pool.backward(dpool)
        dz_conv = dact * (self.z_conv > 0)

        # Backprop through convolution
        self.conv.backward(dz_conv, lr=lr)

        # Dense layer parameter updates
        self.W_fc -= lr * dW_fc
        self.b_fc -= lr * db_fc
        return loss

if __name__ == "__main__":
    np.random.seed(42)
```

```
    model = ToyCNN()

    # Synthetic samples: (1, 8, 8) grids with targets
    X_toy = np.random.randn(4, 1, 8, 8)
    y_toy = np.array([1.0, 0.0, 1.0, 0.0])

    for epoch in range(100):
        total_loss = 0.0
        for i in range(len(X_toy)):
            total_loss += model.train_step(X_toy[i], y_toy[i], lr=0.02)
        if (epoch + 1) % 20 == 0:
            print(f"Epoch {epoch+1:3d} | Loss: {total_loss / len(X_toy)
:.4f}")
```

Listing 2: Complete minimal CNN engine in pure NumPy.

7 Classical and Modern Architectural Paradigms

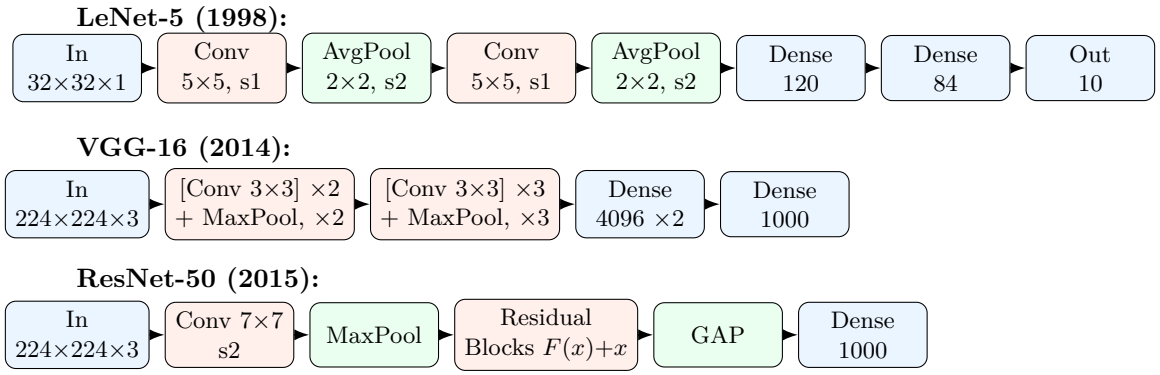


Figure 7: Architectural comparison of LeNet-5, VGG-16, and ResNet-50.

7.1 LeNet-5 (1998)

Developed by Yann LeCun et al. for handwritten digit classification on checks (the MNIST benchmark):

- **Input:** 32×32 grayscale images.
- **Layer C1:** 6 filters of size 5×5, stride 1, valid padding $\rightarrow 28 \times 28 \times 6$ feature maps.
- **Layer S2:** Average pooling with 2×2 windows, stride $2 \rightarrow 14 \times 14 \times 6$ feature maps.
- **Layer C3:** 16 filters of size 5×5, stride 1, valid padding $\rightarrow 10 \times 10 \times 16$ feature maps.
- **Layer S4:** Average pooling with 2×2 windows, stride $2 \rightarrow 5 \times 5 \times 16$ feature maps.
- **Layer C5 (Dense Equivalent):** 120 feature maps of size 1×1.
- **Layer F6:** Dense layer with 84 units using tanh activations.
- **Output:** 10 units using Euclidean Radial Basis Function network (later replaced with Softmax).

7.2 The VGG Philosophy: Small Kernels over Large Ones

In 2014, Simonyan and Zisserman demonstrated that large convolution windows (such as 7×7 or 11×11) are inefficient and sub-optimal. They proposed stacking multiple small 3×3 convolutional layers instead.

- **Receptive Field Equivalence:** Stacking two 3×3 convolutions with unit stride covers the exact same 5×5 spatial receptive field as a single 5×5 kernel. Stacking three 3×3 layers covers a 7×7 field.
- **Fewer Parameters:** Three 3×3 filters across C channels require $3 \times (3^2 \times C^2) = 27C^2$ weights, whereas a single 7×7 filter requires $7^2 \times C^2 = 49C^2$ weights—an immediate 45% reduction in parameter footprint.
- **Increased Non-Linearity:** Three consecutive 3×3 layers incorporate three non-linear activation functions (e.g., ReLU) rather than one, allowing the sub-network to learn more expressive discriminative functions with fewer parameters.

8 Transfer Learning, Data Augmentation, and Practical Workflows

Training modern deep vision models from scratch requires millions of labeled samples and significant compute. Practical workflows solve this by leveraging **Transfer Learning** and **Data Augmentation**.

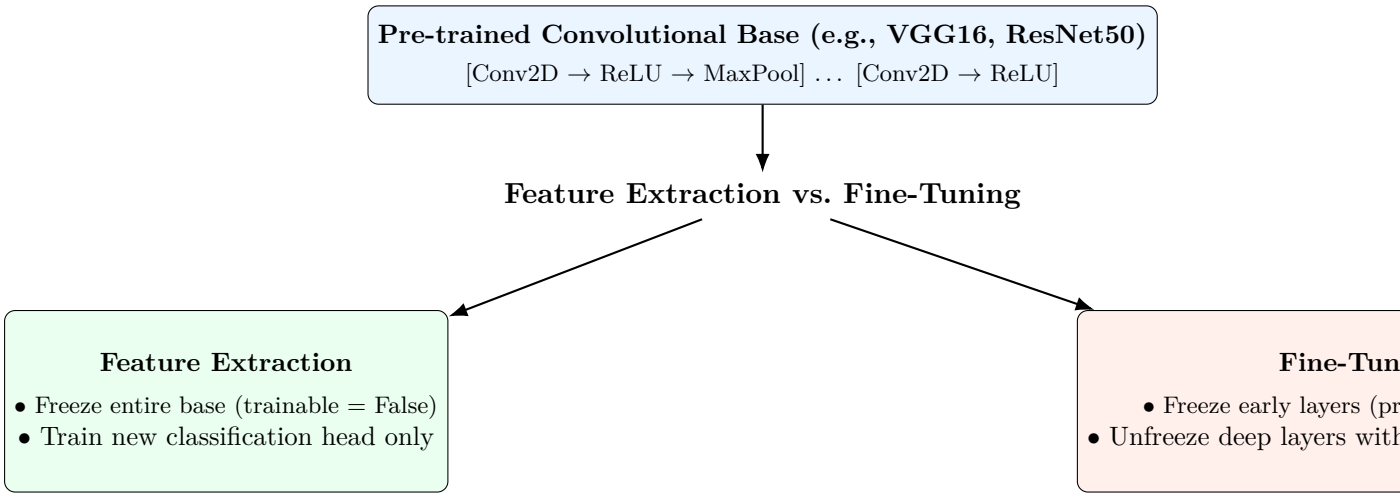


Figure 8: Transfer learning protocols: feature extraction vs. fine-tuning.

8.1 Transfer Learning Protocols

1. **Feature Extraction:** Take a high-capacity model (such as VGG16 or ResNet50) pre-trained on ImageNet. Freeze all weights in the convolutional base (`conv_base.trainable = False`). Append a new classification head (e.g., `Flatten -> Dense(256, relu) -> Dense(1, sigmoid)`) and train only the newly added head. The early layers act as fixed feature extractors for basic visual patterns like edges, textures, and shapes.
2. **Fine-Tuning:** Unfreeze the top convolutional blocks (e.g., block 5 in VGG16) while keeping early layers frozen. Train the unfrozen conv layers alongside the classification head using a very small learning rate (e.g., $\eta = 10^{-5}$ via RMSprop or Adam). This adapts high-level abstract representations to the target dataset without destroying pre-trained weights.

8.2 Data Augmentation

When training on small image datasets, deep CNNs rapidly memorize individual training images. Data augmentation artificially expands the training distribution by applying label-preserving affine transformations:

$$\mathbf{x}_{\text{aug}} = T(\mathbf{x}), \quad T \sim \mathcal{T}_{\text{transforms}} \tag{20}$$

```python
from tensorflow.keras.preprocessing.image import ImageDataGenerator

# Pipeline applying randomized affine transforms on the fly
datagen = ImageDataGenerator(
    rotation_range=30,      # Random rotation between -30 and +30 degrees
    width_shift_range=0.2,  # Horizontal translation
    height_shift_range=0.2, # Vertical translation
    shear_range=0.2,        # Shear mapping
    zoom_range=0.2,         # Random zoom
    horizontal_flip=True,   # Horizontal reflection
    fill_mode='nearest'
)
```

Listing 3: Keras ImageDataGenerator for on-the-fly data augmentation.

By exposing the model to stochastically perturbed variations at every epoch, the network is prevented from overfitting to static pixel alignments, yielding robust classification boundaries on unseen test data.

9 Diagnostic Guide

Symptom	Root Mechanical Cause	Prescribed Architectural Intervention
Rapid memorization: Training accuracy reaches 99% while val accuracy plateaus at ∼70%	Extreme parameter capacity vs. limited training sample size	1. Introduce aggressive Data Augmentation; 2. Add Dropout (0.2–0.5) before dense layers; 3. Replace dense layers with Global Average Pooling; 4. Use a pre-trained base with frozen weights.
Output spatial dimensions collapse prematurely into 0 or negative values	Kernel size exceeds input grid; Excessive unpadded convolutions or aggressive pooling strides	1. Re-evaluate padding ('same' vs. 'valid'); 2. Track spatial size using: $\lfloor (N + 2P - K)/S \rfloor + 1$; 3. Reduce pooling frequency.
Extreme training latency / GPU memory exhaustion	Large fully connected layers attached to wide, high-resolution un-pooled feature maps	1. Replace final Flatten + Dense stack with `GlobalAveragePooling2D`; 2. Reduce batch size or introduce mixed precision; 3. Increase stride in early convolutional stages.
Vanishing gradient in deep convolutions (>15 layers)	Deep stacks of un-residualized layers dampening backward flow	1. Introduce Batch Normalization after convolutions; 2. Add residual identity connections (ResNet); 3. Ensure ReLU / Leaky ReLU replaces Tanh / Sigmoid.

Table 1: Diagnostic guide for common CNN training and architecture issues.

Exercises

1. **Analytical Parameter Count:** Consider an input volume of shape $112 \times 112 \times 64$. Layer 1 applies 128 filters of size 3×3 with stride 1 and padding $P = 1$. Layer 2 is a Max Pooling operation with size 2×2 and stride 2. Compute the exact output tensor dimensions and total trainable parameters for both layers.

2. **Receptive Field Derivation:** A network stacks three convolutional layers without pooling. Layer 1 uses 3×3 filters with stride 1. Layer 2 uses 3×3 filters with stride 2. Layer 3 uses 3×3 filters with stride 1. Derive the effective receptive field of a single neuron in Layer 3 with respect to the original input grid.

3. **Equivariance Proof:** Let T_τ represent an operator that shifts a 2D image spatially by vector τ: $T_\tau(X)(i, j) = X(i - \tau_1, j - \tau_2)$. Prove algebraically that the discrete convolution operator $*$ without boundary clipping commutes with translations, demonstrating translation equivariance:

$$(T_\tau(X)) * W = T_\tau(X * W) \tag{21}$$

Chapter 5

The Transformer Revolution

Attention, Multi-Head Projections,
Positional Geometries, and the Encoder

A Comprehensive Treatment of Attention-Based Architectures

From Seq2Seq Bottlenecks to Self-Attention

Contents

1 The Bottleneck of Recurrence and the Birth of Attention **2**
1.1 The Cross-Attention Mechanism (Bahdanau & Luong) 2

2 Self-Attention: Queries, Keys, Values, and Scaling Dynamics **3**
2.1 Static Word Embeddings vs. Contextual Representations 3
2.2 The Retrieval Analogy: Projections into Query, Key, and Value Spaces 4
2.3 Scaled Dot-Product Attention Equation 4
2.4 The Mathematical Necessity of the Scaling Factor $\sqrt{d_k}$ 5

3 Multi-Head Attention: Disentangling Subspaces and Perspectives **6**
3.1 Mathematical Formulation of Multi-Head Attention (MHA) 6
3.2 Computational Budget Invariance 7

4 Positional Encodings: Injecting Spatial Awareness into Permutation-Equivariant Operators **7**
4.1 The Failure of Naive Indexing Schemes 8
4.2 The Sinusoidal Positional Encoding 8
4.3 Derivation of Relative Linear Transformations 9
4.4 Addition vs. Concatenation 9

5 Normalization in Sequences: Why LayerNorm Prevails over BatchNorm **9**
5.1 The Breakdown of BatchNorm in Sequences 10
5.2 Layer Normalization Mechanics 10

6 The Complete Transformer Encoder Architecture **10**
6.1 1. Input Embedding and Positional Addition 11
6.2 2. The Multi-Head Attention Sub-Layer with Residual Connection 11
6.3 3. The Position-wise Feed-Forward Network (FFN) 11

7 From-Scratch Implementation: A Complete Transformer Encoder Block in Pure NumPy **12**

8 Diagnostic Guide **15**

Exercises **15**

1 The Bottleneck of Recurrence and the Birth of Attention

In Chapter 4, we saw how Recurrent Neural Networks process sequences by iteratively rolling an internal hidden state forward across time:

$$\mathbf{h}_t = \tanh(\mathbf{W}_{xh}\mathbf{x}_t + \mathbf{W}_{hh}\mathbf{h}_{t-1} + \mathbf{b}_h) \tag{1}$$

When applied to sequence-to-sequence tasks like machine translation, this architecture divides cleanly into an **Encoder** and a **Decoder**. The encoder reads an entire input sequence $\mathbf{x}_1, \ldots, \mathbf{x}_T$, updates its hidden state at every step, and emits the final state $\mathbf{h}_T$. This single vector—often termed the **context vector c**—is tasked with compressing the entirety of the input sentence into a static array of numbers.

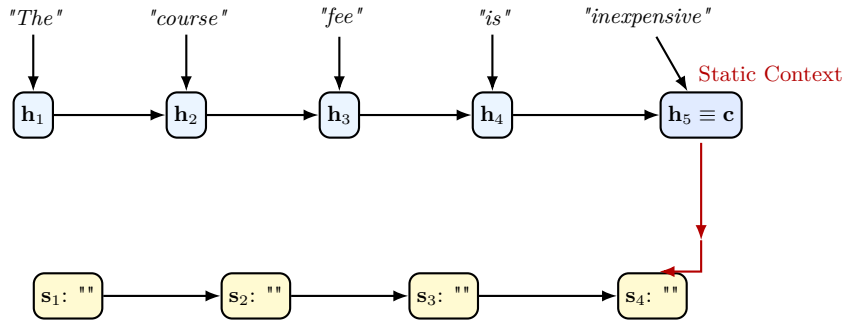


Figure 1: The classic Seq2Seq information bottleneck: the encoder compresses the entire source sentence into a single static context vector $\mathbf{c} = \mathbf{h}_5$, which is then fed to the decoder.

This design exposes a fatal **information bottleneck**. Forcing a fixed-width vector $\mathbf{c} \in \mathbb{R}^H$ (typically $H \in [256, 1024]$) to retain all syntactic, semantic, and relational nuances of sentences longer than 30 tokens degrades performance rapidly. The network suffers from catastrophic forgetting, forcing the decoder to guess based on heavily diluted historical signals.

1.1 The Cross-Attention Mechanism (Bahdanau & Luong)

Dzmitry Bahdanau et al. (2014) and Minh-Thang Luong et al. (2015) removed this bottleneck by eliminating the static context vector. Instead of summarizing the source sequence into a single terminal vector, the encoder retains every intermediate hidden state:

$$\mathbf{H}_{\text{enc}} = [\mathbf{h}_1, \mathbf{h}_2, \ldots, \mathbf{h}_T] \in \mathbb{R}^{T \times H} \tag{2}$$

At each decoding time step i, the decoder maintains an internal state $\mathbf{s}_i$. To determine which source tokens are relevant to emitting the next target word, the model computes an **alignment score** $e_{i,j}$ between the current decoder state $\mathbf{s}_i$ and each encoder hidden state $\mathbf{h}_j$:

$$e_{i,j} = \text{score}(\mathbf{s}_i, \mathbf{h}_j) \tag{3}$$

Formulation	**Score Function**
Bahdanau (Additive)	$\text{score}(\mathbf{s}_i, \mathbf{h}_j) = \mathbf{v}_a^T \tanh(\mathbf{W}_a\mathbf{s}_i + \mathbf{U}_a\mathbf{h}_j)$
Luong (Dot-Product)	$\text{score}(\mathbf{s}_i, \mathbf{h}_j) = \mathbf{s}_i^T\mathbf{h}_j$
Luong (General / Bilinear)	$\text{score}(\mathbf{s}_i, \mathbf{h}_j) = \mathbf{s}_i^T\mathbf{W}_a\mathbf{h}_j$

Table 1: Common alignment scoring formulations.

These scalar scores pass through a softmax function across the source sequence length T to yield normalized **attention weights** $\alpha_{i,j} \in [0, 1]$:

$$\alpha_{i,j} = \frac{\exp(e_{i,j})}{\sum_{k=1}^{T} \exp(e_{i,k})}, \quad \text{such that} \sum_{j=1}^{T} \alpha_{i,j} = 1 \tag{4}$$

The dynamic context vector $\mathbf{c}_i$ is computed as the weighted sum of all encoder hidden states:

$$\mathbf{c}_i = \sum_{j=1}^{T} \alpha_{i,j} \mathbf{h}_j \tag{5}$$

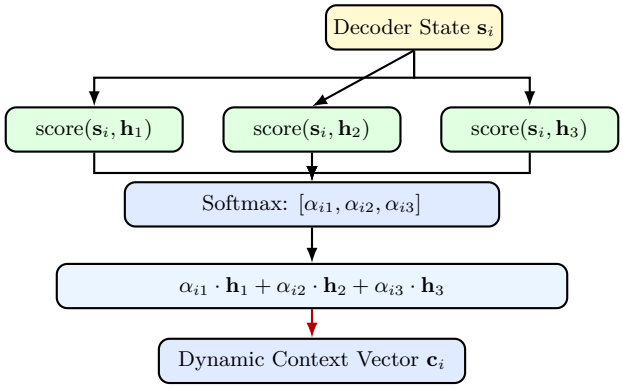


Figure 2: Computing a dynamic context vector $\mathbf{c}_i$ via cross-attention. The decoder state $\mathbf{s}_i$ scores each encoder hidden state, softmax normalizes the scores, and the weighted sum forms the context.

This setup is **inter-sequence (cross) attention**: it measures the alignment between two distinct sequences—the target decoder sequence querying the source encoder sequence.

2 Self-Attention: Queries, Keys, Values, and Scaling Dynamics

Why is self-attention called **"Self"**?

In traditional cross-attention, the query originates from a completely different sequence (the target language decoder), searching across keys and values belonging to the source language encoder. In **Self-Attention**, the mechanism operates strictly **intra-sequence**: a sequence queries itself. Every token in an input sentence attends to every other token within that exact same sentence to construct a richer, context-aware representation.

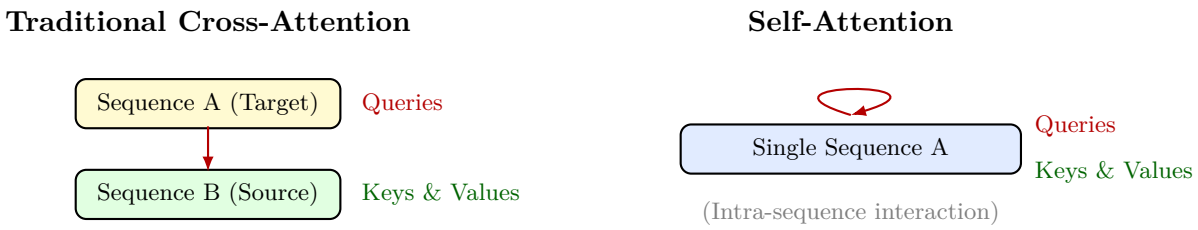


Figure 3: Cross-attention queries a different sequence; self-attention queries the same sequence.

2.1 Static Word Embeddings vs. Contextual Representations

A static embedding layer maps each token to a single vector in a lookup table: $\mathbf{e}_{\text{bank}} \in \mathbb{R}^D$. Under this scheme:

- Sentence 1: `"He deposited money in the bank."`

- Sentence 2: `"The river bank was muddy."`

In both cases, `"bank"` is assigned the identical numerical coordinates, despite referring to a financial institution in the first sentence and a geological formation in the second. Self-attention resolves this by using surrounding tokens to pull the representation of `"bank"` toward financial concepts in Sentence 1, and toward hydrological concepts in Sentence 2.

2.2 The Retrieval Analogy: Projections into Query, Key, and Value Spaces

Given an input sequence represented as an embedding matrix $\mathbf{X} \in \mathbb{R}^{T \times d_{\text{model}}}$, self-attention projects each token into three distinct functional roles via learned weight matrices:

$$\mathbf{Q} = \mathbf{X}\mathbf{W}^Q, \quad \mathbf{W}^Q \in \mathbb{R}^{d_{\text{model}} \times d_k} \tag{6}$$

$$\mathbf{K} = \mathbf{X}\mathbf{W}^K, \quad \mathbf{W}^K \in \mathbb{R}^{d_{\text{model}} \times d_k} \tag{7}$$

$$\mathbf{V} = \mathbf{X}\mathbf{W}^V, \quad \mathbf{W}^V \in \mathbb{R}^{d_{\text{model}} \times d_v} \tag{8}$$

Vector	**Functional Meaning**
Query ($\mathbf{Q}$)	"What attributes am I looking for?"
Key ($\mathbf{K}$)	"What attributes do I possess to offer?"
Value ($\mathbf{V}$)	"What information do I communicate if selected?"

Table 2: Retrieval analogy for the Query, Key, and Value projections.

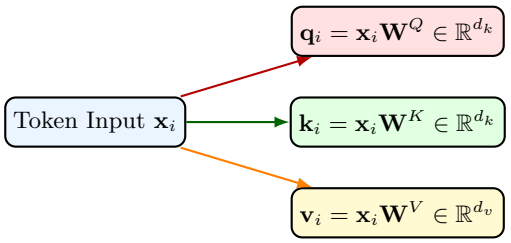


Figure 4: Each input token is projected into three distinct spaces: Query (red), Key (green), and Value (yellow).

2.3 Scaled Dot-Product Attention Equation

To calculate attention across all tokens simultaneously, we compute:

$$\text{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{softmax}\left(\frac{\mathbf{Q}\mathbf{K}^T}{\sqrt{d_k}}\right)\mathbf{V} \tag{9}$$

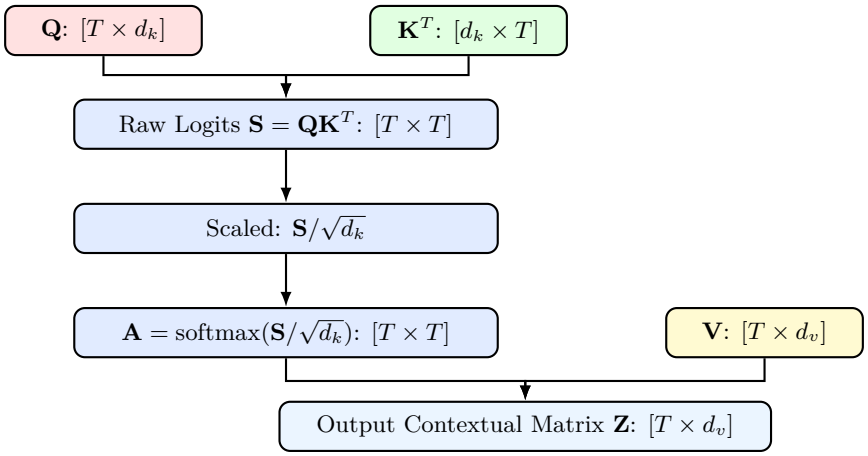


Figure 5: The scaled dot-product attention pipeline: matrix multiply, scale, softmax, then weighted sum with values.

2.4 The Mathematical Necessity of the Scaling Factor $\sqrt{d_k}$

Why do we divide the inner products by $\sqrt{d_k}$?

Assume the components of query vector $\mathbf{q}$ and key vector $\mathbf{k}$ are independent and identically distributed random variables with mean zero and unit variance:

$$\mathbb{E}[q_i] = 0, \quad \text{Var}(q_i) = 1, \quad \mathbb{E}[k_i] = 0, \quad \text{Var}(k_i) = 1 \tag{10}$$

The dot product of these two vectors is a sum of d_k independent random variables:

$$z = \mathbf{q} \cdot \mathbf{k} = \sum_{i=1}^{d_k} q_i k_i \tag{11}$$

Evaluating the mean and variance of this product:

$$\mathbb{E}[z] = \sum_{i=1}^{d_k} \mathbb{E}[q_i k_i] = \sum_{i=1}^{d_k} \mathbb{E}[q_i]\mathbb{E}[k_i] = 0 \tag{12}$$

Using the variance of the product of independent zero-mean random variables:

$$\text{Var}(q_i k_i) = \text{Var}(q_i)\,\text{Var}(k_i) + \text{Var}(q_i)\mathbb{E}[k_i]^2 + \text{Var}(k_i)\mathbb{E}[q_i]^2 = (1)(1) + 0 + 0 = 1 \tag{13}$$

Summing across all d_k dimensions:

$$\text{Var}(z) = \text{Var}\left(\sum_{i=1}^{d_k} q_i k_i\right) = \sum_{i=1}^{d_k} \text{Var}(q_i k_i) = d_k \tag{14}$$

The standard deviation of the raw dot product is:

$$\sigma_z = \sqrt{d_k} \tag{15}$$

As the dimensionality d_k grows (e.g., $d_k = 64$ implies $\sigma_z = 8$; $d_k = 512$ implies $\sigma_z \approx 22.6$), the dot products grow substantially in magnitude.

When large inputs enter the softmax function, the largest value dominates, driving its output probability near 1.0 while pushing all other entries to zero. In these saturated regions, the softmax gradient decays exponentially:

$$\frac{\partial\,\text{softmax}(z)_i}{\partial z_j} = \text{softmax}(z)_i(\delta_{ij} - \text{softmax}(z)_j) \approx 0 \quad \text{when} \;\; \text{softmax}(z)_i \approx 1 \tag{16}$$

This triggers a vanishing gradient that halts optimization early in training. Dividing by $\sqrt{d_k}$ normalizes the variance of the logits back to unit scale ($\text{Var}(z/\sqrt{d_k}) = 1$), ensuring that the softmax function receives inputs with stable, well-behaved distributions.

3 Multi-Head Attention: Disentangling Subspaces and Perspectives

A single self-attention head averages relationships across the sequence into a single set of attention weights. This limits its expressive capacity.

Consider the sentence: *"The man saw the astronomer with a telescope."*

This sentence contains an inherent structural ambiguity:

- **Interpretation A:** Did the man use a telescope to see the astronomer? (Connects `"saw"` $\leftrightarrow$ `"telescope"`)
- **Interpretation B:** Did the astronomer possess the telescope? (Connects `"astronomer"` $\leftrightarrow$ `"telescope"`)

A single attention mechanism must force these distinct syntactic and semantic relationships into a single attention distribution, compromising between them.

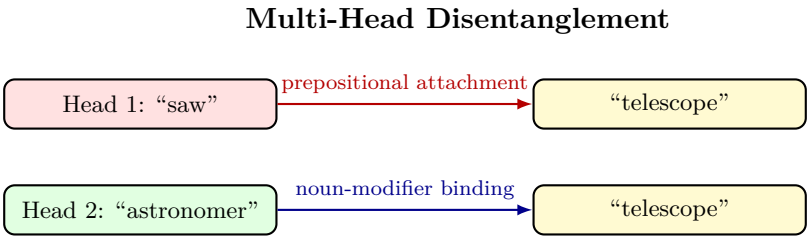


Figure 6: Different attention heads can capture distinct syntactic and semantic relationships in parallel.

3.1 Mathematical Formulation of Multi-Head Attention (MHA)

Rather than computing attention once across the full dimensional space d_{model}, Multi-Head Attention projects the input into h distinct parameter subspaces:

$$\text{head}_i = \text{Attention}\left(\mathbf{Q}\mathbf{W}_i^Q, \mathbf{K}\mathbf{W}_i^K, \mathbf{V}\mathbf{W}_i^V\right), \quad i \in \{1, \ldots, h\} \tag{17}$$

where the projection matrices for each head have shapes:

$$\mathbf{W}_i^Q \in \mathbb{R}^{d_{\text{model}} \times d_k}, \quad \mathbf{W}_i^K \in \mathbb{R}^{d_{\text{model}} \times d_k}, \quad \mathbf{W}_i^V \in \mathbb{R}^{d_{\text{model}} \times d_v} \tag{18}$$

Each head produces an output tensor $\text{head}_i \in \mathbb{R}^{T \times d_v}$. These h independent outputs are concatenated along the channel dimension and passed through a final linear projection matrix $\mathbf{W}^O$:

$$\text{MultiHead}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{Concat}(\text{head}_1, \text{head}_2, \ldots, \text{head}_h)\mathbf{W}^O \tag{19}$$

where $\mathbf{W}^O \in \mathbb{R}^{(h \cdot d_v) \times d_{\text{model}}}$.

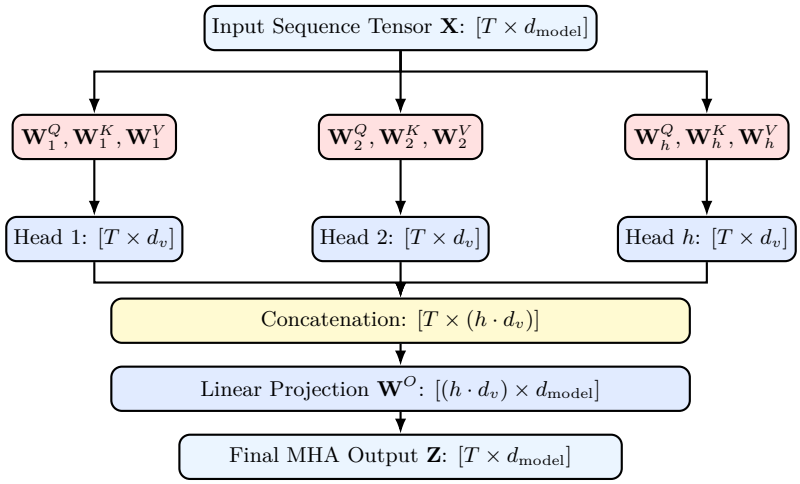


Figure 7: Multi-Head Attention: input is projected into h parallel subspaces, each computed independently, then concatenated and projected back to d_{model}.

3.2 Computational Budget Invariance

In the original Transformer architecture, the model dimensionality is set to $d_{\text{model}} = 512$ with $h = 8$ parallel heads. The key and value dimensions are scaled down accordingly:

$$d_k = d_v = \frac{d_{\text{model}}}{h} = \frac{512}{8} = 64 \tag{20}$$

Examining the computational cost across all heads:

- Computing a single attention head over the full 512 dimensions requires calculating projections of size 512×512, costing $\mathcal{O}(T^2 \cdot 512)$ operations.
- With $h = 8$ heads, each projection maps $512 \to 64$ dimensions. Computing attention across all 8 heads costs:

$$8 \times \mathcal{O}(T^2 \cdot 64) = \mathcal{O}(T^2 \cdot 512) \tag{21}$$

Splitting the architecture into 8 parallel heads adds representational capacity across multiple subspaces without increasing the total floating-point operation (FLOP) count relative to a single full-sized head.

4 Positional Encodings: Injecting Spatial Awareness into Permutation-Equivariant Operators

A pure self-attention layer exhibits **permutation equivariance**. If $\mathbf{P} \in \{0,1\}^{T \times T}$ is an arbitrary permutation matrix that reorders the rows of input matrix $\mathbf{X}$, the self-attention output permutes by that exact same arrangement:

$$\text{Attention}(\mathbf{PXW}^Q, \mathbf{PXW}^K, \mathbf{PXW}^V) = \mathbf{P} \cdot \text{Attention}(\mathbf{XW}^Q, \mathbf{XW}^K, \mathbf{XW}^V) \tag{22}$$

Without an explicit mechanism to break this symmetry, the network processes `"Nitish killed the lion"` and `"The lion killed Nitish"` into identical token-level representations, ignoring grammatical subject-object roles.

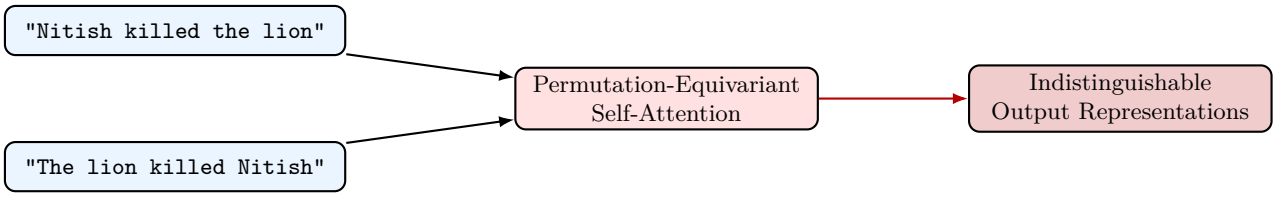


Figure 8: Without positional encoding, self-attention produces identical representations for permuted inputs.

4.1 The Failure of Naive Indexing Schemes

How should position be encoded?

1. **Integer Counting ($pos = 0, 1, 2, \ldots, T-1$):**

   As sequence lengths scale up to large documents (e.g., $T = 100{,}000$), position indices grow unbounded. Adding or concatenating massive integer magnitudes into clean zero-mean embeddings destabilizes gradient updates during backpropagation.

2. **Normalized Scalar Indexing ($pos/T \in [0, 1]$):**

   Dividing indices by the sequence length bounds values safely within $[0, 1]$. However, the step size between adjacent tokens scales inversely with length. In a 4-token sentence, token 2 is located at 0.5; in a 100-token sentence, token 2 sits at 0.02. The network cannot establish consistent interpretations for absolute or relative token distances across varying sequence lengths.

4.2 The Sinusoidal Positional Encoding

Vaswani et al. (2017) addressed this with a non-learned, continuous **Sinusoidal Positional Encoding**. For a token at sequence index $pos \in [0, T-1]$ and channel coordinate $i \in [0, \frac{d_{\text{model}}}{2} - 1]$:

$$PE_{(pos,2i)} = \sin\left(\frac{pos}{10000^{\frac{2i}{d_{\text{model}}}}}\right) \tag{23}$$

$$PE_{(pos,2i+1)} = \cos\left(\frac{pos}{10000^{\frac{2i}{d_{\text{model}}}}}\right) \tag{24}$$

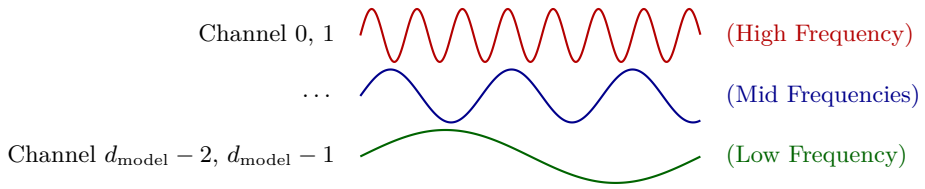


Figure 9: Sinusoidal positional encodings across channel frequencies. Lower channels oscillate rapidly; higher channels change slowly across the sequence.

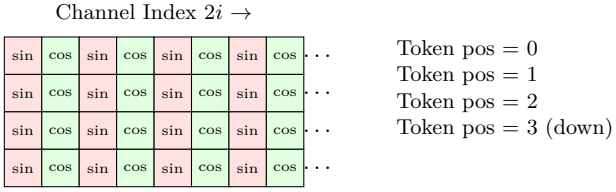


Figure 10: The positional encoding matrix: each row corresponds to a sequence position, each column to a channel. Alternating sin/cos pairs occupy adjacent channels.

The wavelengths form a geometric progression from 2π to $10000 \cdot 2\pi$.

This matches the behavior of a **binary odometer**. The lowest-order bit alternates rapidly on every step $(0, 1, 0, 1)$, while higher-order bits change at progressively slower frequencies. The sinusoidal encoding adapts this principle into a continuous, differentiable space bounded within $[-1, 1]$.

4.3 Derivation of Relative Linear Transformations

A core mathematical property of sinusoidal encodings is that offsets correspond to linear transformations. For any fixed temporal offset k, there exists a linear transformation matrix $\mathbf{M}_k \in \mathbb{R}^{2\times 2}$ that projects the encoding at position pos directly to the encoding at position $pos + k$:

$$PE_{pos+k} = \mathbf{M}_k PE_{pos} \tag{25}$$

Consider an isolated 2D frequency channel $\omega_i = \frac{1}{10000^{2i/d_{\text{model}}}}$:

$$\mathbf{u}_{pos} = \begin{bmatrix} \sin(\omega_i \cdot pos) \\ \cos(\omega_i \cdot pos) \end{bmatrix} \tag{26}$$

Evaluating this vector at offset $pos + k$ using standard angle addition rules:

$$\sin(\omega_i(pos + k)) = \sin(\omega_i \cdot pos)\cos(\omega_i \cdot k) + \cos(\omega_i \cdot pos)\sin(\omega_i \cdot k) \tag{27}$$

$$\cos(\omega_i(pos + k)) = \cos(\omega_i \cdot pos)\cos(\omega_i \cdot k) - \sin(\omega_i \cdot pos)\sin(\omega_i \cdot k) \tag{28}$$

Expressing this linear system as a matrix multiplication:

$$\begin{bmatrix} \sin(\omega_i(pos + k)) \\ \cos(\omega_i(pos + k)) \end{bmatrix} = \begin{bmatrix} \cos(\omega_i \cdot k) & \sin(\omega_i \cdot k) \\ -\sin(\omega_i \cdot k) & \cos(\omega_i \cdot k) \end{bmatrix} \begin{bmatrix} \sin(\omega_i \cdot pos) \\ \cos(\omega_i \cdot pos) \end{bmatrix} \tag{29}$$

The transformation matrix $\mathbf{M}_k$ is an orthogonal rotation matrix dependent solely on the step offset k, independent of the absolute token index pos. This allows the attention mechanism to learn relative positional relationships directly through linear operations.

4.4 Addition vs. Concatenation

Why do we add the positional encoding to the token embedding ($\mathbf{X}$+$\mathbf{PE}$) rather than concatenating them ($[\mathbf{X} \,\|\, \mathbf{PE}]$)?

Concatenating an embedding of size d_{model} with an encoding of size d_{model} expands the channel width to $2 \cdot d_{\text{model}}$. This doubles the row dimension of every weight matrix in the subsequent multi-head attention and feedforward layers, substantially increasing parameter counts and training latency.

By projecting inputs into a high-dimensional vector space (e.g., $d_{\text{model}} = 512$), semantic features and sinusoidal components populate approximately orthogonal directions. Adding them injects spatial coordinates directly into the representation without expanding parameter budgets or channel dimensions.

5 Normalization in Sequences: Why LayerNorm Prevails over BatchNorm

In Chapter 2, we examined Batch Normalization (BatchNorm). Despite its success in convolutional vision models, **BatchNorm fails when applied to variable-length sequences**.

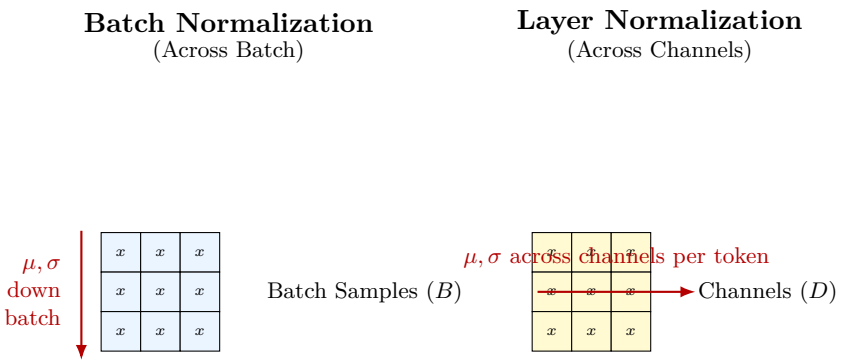


Figure 11: BatchNorm computes statistics down the batch axis; LayerNorm computes statistics across the channel axis independently for each token.

5.1 The Breakdown of BatchNorm in Sequences

1. **Zero-Padding Artifacts:** Batched sequences are padded with zero tokens to match the longest item in the batch. When BatchNorm calculates statistics across the batch axis at a given time step t, these artificial padding tokens distort the calculated means and variances.

2. **Inference Instability:** If a test sequence is longer than any sequence seen during training, BatchNorm has no valid running statistics μ_{run} or σ^2_{run} for those extended time steps, causing inference predictions to break down.

3. **Small Batch Sensitivity:** In large models, memory constraints often restrict batch sizes to small numbers (e.g., $B \in [2, 8]$). Small batch sizes introduce high variance into batch statistics, destabilizing optimization.

5.2 Layer Normalization Mechanics

Jimmy Lei Ba, Jamie Ryan Kiros, and Geoffrey Hinton (2016) introduced **Layer Normalization (LayerNorm)**. Instead of computing statistics across the batch dimension, LayerNorm calculates the mean and variance across the feature channels of each token independently.

Given a single token activation vector $\mathbf{z} \in \mathbb{R}^{d_{\text{model}}}$ at an arbitrary batch and temporal position:

$$\mu = \frac{1}{d_{\text{model}}} \sum_{j=1}^{d_{\text{model}}} z_j \tag{30}$$

$$\sigma^2 = \frac{1}{d_{\text{model}}} \sum_{j=1}^{d_{\text{model}}} (z_j - \mu)^2 \tag{31}$$

$$\hat{\mathbf{z}} = \frac{\mathbf{z} - \mu}{\sqrt{\sigma^2 + \epsilon}} \tag{32}$$

$$\mathbf{y} = \boldsymbol{\gamma} \odot \hat{\mathbf{z}} + \boldsymbol{\beta} \tag{33}$$

where $\boldsymbol{\gamma}, \boldsymbol{\beta} \in \mathbb{R}^{d_{\text{model}}}$ are learnable scale and shift parameters.

Because LayerNorm normalizes each token vector using only its own internal channels, its calculations are unaffected by sequence length, batch size, or zero-padding in neighboring samples.

6 The Complete Transformer Encoder Architecture

The complete Transformer Encoder is built by stacking $N = 6$ identical encoder layers sequentially.

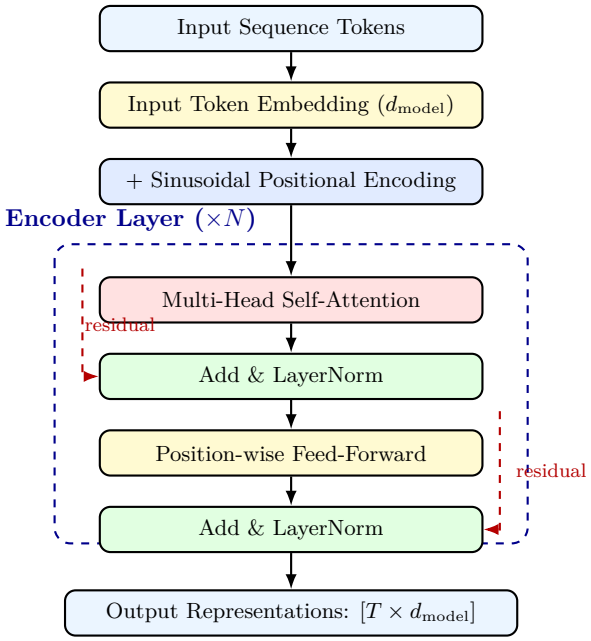


Figure 12: The complete Transformer Encoder. Each of the N identical layers contains a Multi-Head Self-Attention sub-layer and a Position-wise Feed-Forward sub-layer, each wrapped with a residual connection and LayerNorm.

6.1 1. Input Embedding and Positional Addition

Input token sequences pass through an embedding lookup matrix to produce dense vectors of dimension $d_{\text{model}} = 512$. These vectors are combined directly with matching sinusoidal positional encoding tensors:

$$\mathbf{X}^{(0)} = \text{Embedding}(\mathbf{X}) + \mathbf{PE}, \quad \mathbf{X}^{(0)} \in \mathbb{R}^{T \times d_{\text{model}}} \tag{34}$$

6.2 2. The Multi-Head Attention Sub-Layer with Residual Connection

The tensor passes into the multi-head self-attention module. To prevent vanishing gradients across deep layer stacks, the input is added back to the sub-layer output via a **residual (skip) connection** before being normalized:

$$\mathbf{Z}^{(l)} = \text{LayerNorm}\left(\mathbf{X}^{(l-1)} + \text{MultiHead}(\mathbf{X}^{(l-1)}, \mathbf{X}^{(l-1)}, \mathbf{X}^{(l-1)})\right) \tag{35}$$

Note on Architecture Variations: The original 2017 Transformer uses **Post-LN**, applying LayerNorm after the residual addition. Many modern architectures adopt **Pre-LN**, applying normalization directly to the input of each sub-layer ($\mathbf{X} + \text{Sublayer}(\text{LayerNorm}(\mathbf{X}))$), which improves gradient stability and eases warm-up requirements in very deep stacks.

6.3 3. The Position-wise Feed-Forward Network (FFN)

The normalized attention representations pass into a two-layer, fully connected **Position-wise Feed-Forward Network**. This MLP operates on every sequence token position independently and identically:

$$\text{FFN}(\mathbf{z}) = \max(0, \mathbf{z}\mathbf{W}_1 + \mathbf{b}_1)\mathbf{W}_2 + \mathbf{b}_2 \tag{36}$$

where parameter dimensions expand and contract according to:

$$\mathbf{W}_1 \in \mathbb{R}^{d_{\text{model}} \times d_{\text{ff}}}, \quad \mathbf{W}_2 \in \mathbb{R}^{d_{\text{ff}} \times d_{\text{model}}}, \quad d_{\text{ff}} = 4 \cdot d_{\text{model}} = 2048 \tag{37}$$

While multi-head attention handles communication between different tokens, the FFN processes representations within each token independently, projecting them into a higher-dimensional space (2048) before returning to d_{model}.

The FFN output is routed through a second residual connection and normalization step:

$$\mathbf{X}^{(l)} = \text{LayerNorm}\left(\mathbf{Z}^{(l)} + \text{FFN}(\mathbf{Z}^{(l)})\right) \tag{38}$$

The tensor maintains its dimensions ($\mathbb{R}^{T \times d_{\text{model}}}$) as it exits the sub-layer, allowing it to pass directly into subsequent encoder blocks.

7 From-Scratch Implementation: A Complete Transformer Encoder Block in Pure NumPy

Below is a self-contained, vectorized implementation of a complete Transformer Encoder layer written from scratch in NumPy, including multi-head attention, sinusoidal positional encoding, layer normalization, and the position-wise feed-forward network.

```python
import numpy as np

def get_sinusoidal_positional_encoding(seq_len: int, d_model: int) -> np.ndarray:
    """
    Computes sinusoidal positional encodings up to seq_len for a d_model channel space.
    """
    pe = np.zeros((seq_len, d_model))
    positions = np.arange(seq_len)[:, np.newaxis]  # (T, 1)

    # Exponent for geometric frequency scaling: 10000^(2i / d_model)
    div_term = np.exp(np.arange(0, d_model, 2) * -(np.log(10000.0) / d_model))

    pe[:, 0::2] = np.sin(positions * div_term)
    pe[:, 1::2] = np.cos(positions * div_term)
    return pe


class LayerNormalization:
    """
    Layer Normalization applied across feature channels independently for each token.
    """
    def __init__(self, d_model: int, eps: float = 1e-5):
        self.gamma = np.ones(d_model)
        self.beta = np.zeros(d_model)
        self.eps = eps

    def forward(self, x: np.ndarray) -> np.ndarray:
        # x shape: (Batch, Seq_Len, d_model)
        mean = np.mean(x, axis=-1, keepdims=True)
        variance = np.var(x, axis=-1, keepdims=True)
        x_norm = (x - mean) / np.sqrt(variance + self.eps)
        return self.gamma * x_norm + self.beta


class MultiHeadAttention:
    """
```

```
    Multi-Head Attention using parallel matrix projections.
    """
    def __init__(self, d_model: int = 512, num_heads: int = 8):
        assert d_model % num_heads == 0, "d_model must be divisible by num_heads"
        self.d_model = d_model
        self.h = num_heads
        self.d_k = d_model // num_heads

        # Xavier Normal parameter initializations
        scale = np.sqrt(2.0 / (d_model + d_model))
        self.W_q = np.random.randn(d_model, d_model) * scale
        self.W_k = np.random.randn(d_model, d_model) * scale
        self.W_v = np.random.randn(d_model, d_model) * scale
        self.W_o = np.random.randn(d_model, d_model) * scale

    def forward(self, x: np.ndarray, mask: np.ndarray = None) -> np.ndarray:
        B, T, _ = x.shape

        # 1. Linear projections: (B, T, d_model)
        Q = np.dot(x, self.W_q)
        K = np.dot(x, self.W_k)
        V = np.dot(x, self.W_v)

        # 2. Reshape and transpose into heads: (B, h, T, d_k)
        Q = Q.reshape(B, T, self.h, self.d_k).transpose(0, 2, 1, 3)
        K = K.reshape(B, T, self.h, self.d_k).transpose(0, 2, 1, 3)
        V = V.reshape(B, T, self.h, self.d_k).transpose(0, 2, 1, 3)

        # 3. Scaled dot-product logits: (B, h, T, T)
        scores = np.matmul(Q, K.transpose(0, 1, 3, 2)) / np.sqrt(self.d_k)

        if mask is not None:
            scores = np.where(mask == 0, -1e9, scores)

        # 4. Softmax over target sequence axis
        exp_scores = np.exp(scores - np.max(scores, axis=-1, keepdims=True))
        attn_weights = exp_scores / np.sum(exp_scores, axis=-1, keepdims=True)

        # 5. Weighted values aggregation: (B, h, T, d_k)
        head_out = np.matmul(attn_weights, V)

        # 6. Recombine heads: (B, T, d_model)
        concat_out = head_out.transpose(0, 2, 1, 3).reshape(B, T, self.d_model)

        # 7. Final output projection
        return np.dot(concat_out, self.W_o)


class PositionwiseFeedForward:
    """
    Two-layer MLP applied to each token position independently:
    Linear(d_model -> d_ff) -> ReLU -> Linear(d_ff -> d_model)
    """
    def __init__(self, d_model: int = 512, d_ff: int = 2048):
```

```
        scale1 = np.sqrt(2.0 / (d_model + d_ff))
        scale2 = np.sqrt(2.0 / (d_ff + d_model))
        self.W1 = np.random.randn(d_model, d_ff) * scale1
        self.b1 = np.zeros(d_ff)
        self.W2 = np.random.randn(d_ff, d_model) * scale2
        self.b2 = np.zeros(d_model)

    def forward(self, x: np.ndarray) -> np.ndarray:
        # Affine 1 + ReLU activation
        hidden = np.maximum(0, np.dot(x, self.W1) + self.b1)
        # Affine 2 projection back to d_model
        return np.dot(hidden, self.W2) + self.b2


class TransformerEncoderBlock:
    """
    A single Post-LN Transformer Encoder layer:
    x -> MHA -> Add & LayerNorm -> FFN -> Add & LayerNorm
    """
    def __init__(self, d_model: int = 512, num_heads: int = 8, d_ff: int =
    2048):
        self.mha = MultiHeadAttention(d_model=d_model, num_heads=num_heads
    )
        self.norm1 = LayerNormalization(d_model=d_model)
        self.ffn = PositionwiseFeedForward(d_model=d_model, d_ff=d_ff)
        self.norm2 = LayerNormalization(d_model=d_model)

    def forward(self, x: np.ndarray, mask: np.ndarray = None) -> np.
    ndarray:
        # Sub-layer 1: Multi-Head Attention with residual skip and
    LayerNorm
        attn_out = self.mha.forward(x, mask=mask)
        x = self.norm1.forward(x + attn_out)

        # Sub-layer 2: Feed-Forward Network with residual skip and
    LayerNorm
        ffn_out = self.ffn.forward(x)
        x = self.norm2.forward(x + ffn_out)
        return x


if __name__ == "__main__":
    np.random.seed(42)

    # Simulation hyperparameters
    batch_size = 2
    seq_length = 4  # e.g., ["How", "are", "you", "<pad>"]
    d_model = 512

    # 1. Random word embeddings
    token_embeddings = np.random.randn(batch_size, seq_length, d_model)

    # 2. Add sinusoidal positional encoding
    pos_encoding = get_sinusoidal_positional_encoding(seq_length, d_model)
    x = token_embeddings + pos_encoding  # Shape: (2, 4, 512)

    # 3. Create boolean padding mask (1 for real tokens, 0 for padding
    tokens)
    mask = np.ones((batch_size, 1, 1, seq_length))
```

```
    mask[0, :, :, 3] = 0  # Mark the final token of sample 0 as padding

    # 4. Instantiate and execute the encoder block
    encoder_block = TransformerEncoderBlock(d_model=512, num_heads=8, d_ff=2048)
    output = encoder_block.forward(x, mask=mask)

    print("Input Tensor Shape: ", x.shape)
    print("Output Tensor Shape:", output.shape)
    assert output.shape == (batch_size, seq_length, d_model), "Dimension mismatch!"
```

Listing 1: Complete Transformer Encoder block implemented from scratch in NumPy.

8 Diagnostic Guide

Symptom	**Mechanical Root Cause**	**Resolution Protocol**
Attention distributions collapse into one-hot selections early in training	Attention logits grow large, saturating softmax and zeroing gradients	1. Ensure logits are scaled by $1/\sqrt{d_k}$ before softmax; 2. Verify query/key weights use Xavier/He normal scaling; 3. Lower learning rate or introduce warm-up schedules.
Model ignores word order; identical outputs for reversed input sequences	Positional encodings omitted or scaled down relative to token embeddings	1. Check that positional encodings are added to embeddings; 2. Scale token embeddings by $\sqrt{d_{\text{model}}}$ prior to adding PE, ensuring embedding norms match positional scales.
Loss evaluates to `NaN` during early training iterations	Post-LN gradient instability in deep stacks; learning rate is too high without warm-up	1. Switch from Post-LN to Pre-LN architecture; 2. Implement a linear learning rate warmup phase; 3. Apply gradient clipping to the norm of all parameters.
Padding tokens bleed into output representations	Masking missing or improperly broadcast across attention heads	1. Apply attention masking: set padded logit entries to -10^9 prior to computing softmax; 2. Verify padding mask broadcast shape: $(B, 1, 1, T)$.

Table 3: Diagnostic guide for common Transformer training and architecture issues.

Exercises

1. **The Role of Scaling in Softmax:** Let $\mathbf{q}, \mathbf{k} \in \mathbb{R}^{d_k}$ be vectors whose entries are sampled from $\mathcal{N}(0, 1)$. Compute the variance of their dot product $\mathbf{q}^T\mathbf{k}$ when $d_k = 64$ and $d_k = 1024$. Explain how dividing by $\sqrt{d_k}$ preserves variance across dimension choices, and trace what happens to the softmax gradient vector when this scaling is omitted.

2. **Computational Complexity: Attention vs. Recurrence:** Compare the computational complexity and parallelization capacity of an unrolled SimpleRNN layer versus a Self-Attention

layer processing a sequence of length T with hidden state dimension d. Under what sequence length condition T does Self-Attention become more computationally expensive per forward pass than an RNN?

3. **The Pre-LN vs. Post-LN Gradient Path:** Write down the algebraic expressions for the residual update in Post-LN ($\mathbf{x}_l = \text{LayerNorm}(\mathbf{x}_{l-1} + \mathcal{F}(\mathbf{x}_{l-1}))$) and Pre-LN ($\mathbf{x}_l = \mathbf{x}_{l-1} + \mathcal{F}(\text{LayerNorm}(\mathbf{x}_{l-1}))$). Trace the backward gradient path $\frac{\partial \mathbf{x}_L}{\partial \mathbf{x}_0}$ for both variants, and show why Pre-LN maintains a clean identity highway that stabilizes optimization in deep networks without needing aggressive warm-up schedules.

Chapter 4

Sequential Dynamics

Recurrence, The Vanishing Gradient Dilemma,
LSTMs, GRUs, and Embeddings

A Comprehensive Treatment of Recurrent Neural Networks

From Vanilla RNNs to Gated Architectures

Contents

1 From Spatial Feeds to Temporal Streams: The Geometry of Sequences **2**
1.1 Vectorizing Language: Discrete Indices to Continuous Embeddings 2

2 The Vanilla Recurrent Neural Network (SimpleRNN) **3**
2.1 Unrolling Through Time 3
2.2 Parameter Counting 4

3 Backpropagation Through Time (BPTT) and The Vanishing Gradient Proof **4**
3.1 Analytical Derivation of the Gradient 5
3.2 The Mathematical Mechanics of the Gradient Breakdown 5
3.3 Mitigating Exploding Gradients: Gradient Clipping 6

4 Long Short-Term Memory (LSTM): Engineering an Additive Error Highway **6**
4.0.1 1. The Forget Gate ($\mathbf{f}_t$) 7
4.0.2 2. The Input Gate ($\mathbf{i}_t$) and Candidate Cell State ($\tilde{\mathbf{C}}_t$) 7
4.0.3 3. Updating the Cell State ($\mathbf{C}_t$) 7
4.0.4 4. The Output Gate ($\mathbf{o}_t$) and Final Hidden State ($\mathbf{h}_t$) 7
4.1 Why the Cell State Prevents Vanishing Gradients 8
4.2 LSTM Parameter Counting 8

5 The Gated Recurrent Unit (GRU): Streamlined Gating **8**
5.0.1 1. The Reset Gate ($\mathbf{r}_t$) 9
5.0.2 2. The Update Gate ($\mathbf{z}_t$) 9
5.0.3 3. The Candidate Hidden State ($\tilde{\mathbf{h}}_t$) 9
5.0.4 4. The Final Hidden State ($\mathbf{h}_t$) 9
5.1 GRU Parameter Counting 9

6 Stacked (Deep) Recurrent Architectures **10**
6.1 Hierarchical Representation in Language 10
6.2 The Role of `return_sequences` 10

7 From Scratch Implementation: A Minimal Next-Word Language Model Engine **10**
7.1 Production Verification with Stacked Keras Models 13

8 Architectural Comparison and Diagnostic Guide **14**
8.1 Troubleshooting Sequence Models 14

Exercises **15**

1 From Spatial Feeds to Temporal Streams: The Geometry of Sequences

In Chapters 1 through 3, our models operated under an assumption of exchangeability or fixed spatial bounds: an input vector $\mathbf{x} \in \mathbb{R}^d$ or image tensor $\mathbf{X} \in \mathbb{R}^{H \times W \times C}$ had a static dimensionality, and the computational graph processed the entire input in a single forward pass.

Real-world phenomena—natural language, speech signals, financial ticks, protein sequences—are fundamentally dynamic, ordered streams:

$$\mathbf{x}_1, \mathbf{x}_2, \mathbf{x}_3, \ldots, \mathbf{x}_T \tag{1}$$

Processing sequential data with standard Multi-Layer Perceptrons (MLPs) or Convolutional Neural Networks (CNNs) exposes critical architectural shortcomings:

1. **Arbitrary and Variable Sequence Lengths:** A sentence may contain four tokens or four hundred. Standard dense layers require a strictly fixed input dimension d; they cannot dynamically grow or shrink their weight matrices.
2. **Positional Rigidness and Temporal Invariance:** If a word appears at index 1 versus index 20, a feedforward network treats it through entirely disjoint weight parameters. There is no native parameter sharing across time.
3. **Loss of Relational Order:** Simple pooling or bag-of-words encodings destroy token order, treating `"dog bites man"` and `"man bites dog"` identically.

To model sequences, a network must process tokens iteratively while maintaining an evolving internal state—a **memory vector** that aggregates historical context across arbitrary temporal distances.

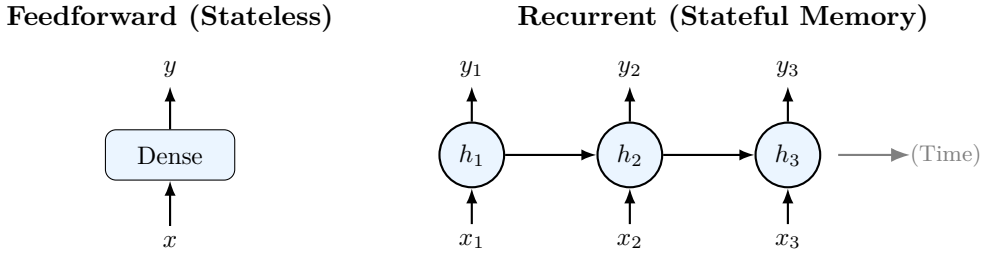


Figure 1: Feedforward processing (left) treats each input independently. Recurrent processing (right) threads a hidden state h_t through time, giving the model memory.

1.1 Vectorizing Language: Discrete Indices to Continuous Embeddings

Raw text enters a system as discrete symbols. The first step maps each word into an integer index using a fixed vocabulary dictionary $\mathcal{V}$ of size V:

$$\text{"the"} \to 1, \quad \text{"course"} \to 2, \quad \text{"fee"} \to 3 \ldots \tag{2}$$

Representing token $w \in \mathcal{V}$ as a sparse one-hot vector $\mathbf{e}_w \in \{0, 1\}^V$ produces orthogonal, equidistant representations: the dot product between any two distinct one-hot vectors is identically zero, discarding all semantic relationships.

An **Embedding Layer** projects discrete tokens into a continuous geometric subspace $\mathbb{R}^D$ (where $D \ll V$) via a learned lookup matrix $\mathbf{E} \in \mathbb{R}^{V \times D}$:

$$\mathbf{x}_t = \mathbf{e}_{w_t}^T \mathbf{E} = \mathbf{E}[w_t, :] \tag{3}$$

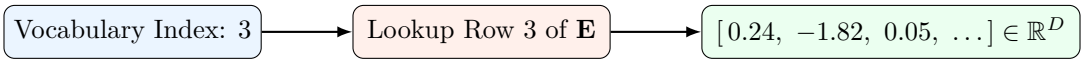


Figure 2: An embedding layer performs a table lookup: the integer token index directly indexes row w_t of the embedding matrix $\mathbf{E}$.

Rather than executing an expensive matrix multiplication against a sparse vector, an embedding operation acts as direct pointer arithmetic (slicing row w_t). During backpropagation, the gradients with respect to $\mathbf{E}$ accumulate strictly into the specific rows active during that training batch.

2 The Vanilla Recurrent Neural Network (SimpleRNN)

A Recurrent Neural Network (RNN) introduces a directed cycle in its computational graph. At each discrete time step $t \in \{1, \ldots, T\}$, the network consumes an incoming feature vector $\mathbf{x}_t \in \mathbb{R}^D$ alongside its own previous hidden state $\mathbf{h}_{t-1} \in \mathbb{R}^H$, producing an updated hidden state $\mathbf{h}_t \in \mathbb{R}^H$:

$$\mathbf{h}_t = \tanh(\mathbf{W}_{xh}\mathbf{x}_t + \mathbf{W}_{hh}\mathbf{h}_{t-1} + \mathbf{b}_h) \tag{4}$$

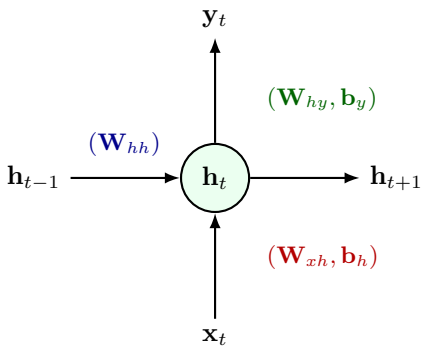


Figure 3: The vanilla RNN cell: the hidden state $\mathbf{h}_t$ is computed from the current input $\mathbf{x}_t$ and the previous hidden state $\mathbf{h}_{t-1}$, and can be projected to an output $\mathbf{y}_t$.

- $\mathbf{W}_{xh} \in \mathbb{R}^{H \times D}$: Input-to-hidden projection matrix.
- $\mathbf{W}_{hh} \in \mathbb{R}^{H \times H}$: Hidden-to-hidden recurrent state transition matrix.
- $\mathbf{b}_h \in \mathbb{R}^H$: Hidden layer bias vector.
- tanh: Non-linear squashing activation restricting hidden states to $(-1, 1)$.

If the task requires predicting an output at time t (e.g., sequence-to-sequence tagging or next-token language modeling), we route $\mathbf{h}_t$ through an affine projection to compute logits:

$$\mathbf{z}_t = \mathbf{W}_{hy}\mathbf{h}_t + \mathbf{b}_y, \quad \hat{\mathbf{y}}_t = \text{softmax}(\mathbf{z}_t) \tag{5}$$

where $\mathbf{W}_{hy} \in \mathbb{R}^{K \times H}$ and $\mathbf{b}_y \in \mathbb{R}^K$ for a classification task over K target categories.

2.1 Unrolling Through Time

Because the parameters $\{\mathbf{W}_{xh}, \mathbf{W}_{hh}, \mathbf{W}_{hy}, \mathbf{b}_h, \mathbf{b}_y\}$ are shared across every time step, we can unroll the recursive loop into a sequence of identical operations:

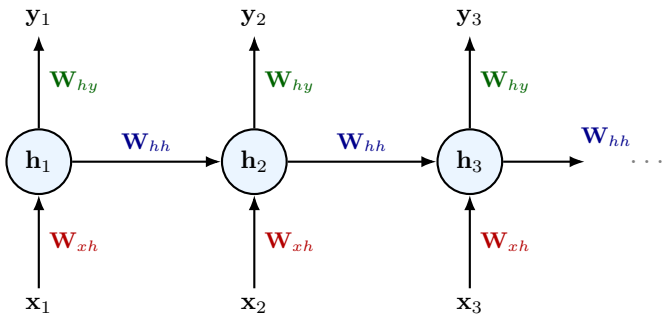


Figure 4: An RNN unrolled across three time steps. The same weight matrices are reused at every step.

An RNN unrolled across T steps is functionally equivalent to a T-layer feedforward network with two constraints:

1. The weights at each time depth are constrained to be identical.

2. New external inputs $\mathbf{x}_t$ are injected at every layer.

2.2 Parameter Counting

In a recurrent layer with hidden dimension H consuming input embeddings of dimension D, the trainable parameter tally is:

$$\text{Parameters} = \underbrace{(D \times H)}_{\mathbf{W}_{xh}} + \underbrace{(H \times H)}_{\mathbf{W}_{hh}} + \underbrace{H}_{\mathbf{b}_h} = H(D + H + 1) \tag{6}$$

- **Concrete Example:** For an input sequence with embedding dimension $D = 32$ routed into an RNN layer with $H = 5$ units:

$$\mathbf{W}_{xh} \in \mathbb{R}^{5\times 32} \implies 160 \text{ weights} \tag{7}$$
$$\mathbf{W}_{hh} \in \mathbb{R}^{5\times 5} \implies 25 \text{ weights} \tag{8}$$
$$\mathbf{b}_h \in \mathbb{R}^{5} \implies 5 \text{ biases} \tag{9}$$
$$\text{Total} = 160 + 25 + 5 = 190 \text{ parameters} \tag{10}$$

The sequence length T does not alter parameter counts; the identical 190 parameters process sequences whether $T = 10$ or $T = 10{,}000$.

3 Backpropagation Through Time (BPTT) and The Vanishing Gradient Proof

Training an RNN requires minimizing a cumulative loss function over the sequence:

$$L = \sum_{t=1}^{T} L_t(\hat{\mathbf{y}}_t, \mathbf{y}_t) \tag{11}$$

Calculating gradients with respect to the shared weight matrices requires backpropagating backward through time—an algorithm known as **Backpropagation Through Time (BPTT)**.

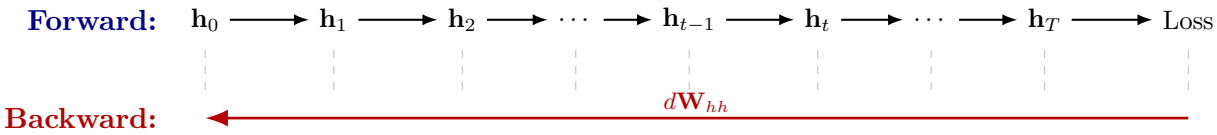


Figure 5: Forward pass through time (top) and the backward BPTT pass (bottom). The recurrent weight gradient accumulates contributions from every time step.

3.1 Analytical Derivation of the Gradient

Consider computing the derivative of the loss at the final time step L_T with respect to the recurrent weight matrix $\mathbf{W}_{hh}$. Because $\mathbf{W}_{hh}$ participates in every step from $t = 1$ up to T, changing $\mathbf{W}_{hh}$ impacts L_T through multiple recurrent paths:

$$\frac{\partial L_T}{\partial \mathbf{W}_{hh}} = \sum_{k=1}^{T} \frac{\partial L_T}{\partial \mathbf{h}_T} \frac{\partial \mathbf{h}_T}{\partial \mathbf{h}_k} \frac{\partial \mathbf{h}_k}{\partial \mathbf{W}_{hh}} \tag{12}$$

By the multivariable chain rule, the temporal Jacobian term decomposes into a continuous product of state transitions:

$$\frac{\partial \mathbf{h}_T}{\partial \mathbf{h}_k} = \prod_{j=k+1}^{T} \frac{\partial \mathbf{h}_j}{\partial \mathbf{h}_{j-1}} \tag{13}$$

Let us inspect the individual single-step Jacobian matrix $\frac{\partial \mathbf{h}_j}{\partial \mathbf{h}_{j-1}}$. Given:

$$\mathbf{h}_j = \tanh(\mathbf{z}_j), \quad \mathbf{z}_j = \mathbf{W}_{xh}\mathbf{x}_j + \mathbf{W}_{hh}\mathbf{h}_{j-1} + \mathbf{b}_h \tag{14}$$

The Jacobian evaluates to:

$$\frac{\partial \mathbf{h}_j}{\partial \mathbf{h}_{j-1}} = \operatorname{diag}\left(1 - \mathbf{h}_j^2\right) \mathbf{W}_{hh}^T \tag{15}$$

where $\operatorname{diag}\left(1 - \mathbf{h}_j^2\right)$ is a diagonal matrix containing the local derivatives of tanh evaluated at each unit.

3.2 The Mathematical Mechanics of the Gradient Breakdown

Substituting the individual Jacobians back into the sequence product yields:

$$\frac{\partial \mathbf{h}_T}{\partial \mathbf{h}_k} = \prod_{j=k+1}^{T} \operatorname{diag}\left(1 - \mathbf{h}_j^2\right) \mathbf{W}_{hh}^T \tag{16}$$

Let us analyze what happens as the temporal gap $(T - k)$ grows large:

1. **The** tanh **Compression Factor:** The derivative of $\tanh(z)$ is bounded strictly within $[0, 1]$, and evaluates to near zero whenever neurons saturate ($|z| > 2$). The elements of the diagonal matrix satisfy:
$$\left|\operatorname{diag}\left(1 - \mathbf{h}_j^2\right)_{ii}\right| \leq 1 \tag{17}$$

2. **Spectral Properties of $\mathbf{W}_{hh}$:** Let $\lambda_{\max}$ denote the largest absolute eigenvalue (spectral radius) of the recurrent transition matrix $\mathbf{W}_{hh}$. Repeated multiplication of the weight matrix scales the product proportionally to $(\lambda_{\max})^{T-k}$:

- **If $\lambda_{\max} < 1$:** The magnitude of the matrix product decays exponentially toward zero:

$$\lim_{(T-k)\to\infty} \left\| \frac{\partial \mathbf{h}_T}{\partial \mathbf{h}_k} \right\| = 0 \tag{18}$$

  Vanishing Gradient: Signals from distant time steps $k \ll T$ vanish entirely. The network cannot learn long-range temporal dependencies (e.g., connecting a subject at token 1 to a verb at token 50).

- **If $\lambda_{\max} > 1$:** The matrix product can grow exponentially:

$$\lim_{(T-k)\to\infty} \left\| \frac{\partial \mathbf{h}_T}{\partial \mathbf{h}_k} \right\| = \infty \tag{19}$$

  Exploding Gradient: Gradients blow up into numerical instability (`NaN`), causing parameter updates to oscillate wildly and break optimization.

3.3 Mitigating Exploding Gradients: Gradient Clipping

While vanishing gradients stem from structural model decay, exploding gradients can be mitigated using **norm-based gradient clipping**:

$$\mathbf{g} \leftarrow \begin{cases} \mathbf{g} & \text{if } \|\mathbf{g}\| \le c \\ \frac{c}{\|\mathbf{g}\|}\mathbf{g} & \text{if } \|\mathbf{g}\| > c \end{cases} \tag{20}$$

```python
import numpy as np

def clip_gradients(gradients: list[np.ndarray], max_norm: float = 1.0) -> list[np.ndarray]:
    total_norm = np.sqrt(sum(np.sum(g ** 2) for g in gradients))
    if total_norm > max_norm:
        scale = max_norm / (total_norm + 1e-8)
        return [g * scale for g in gradients]
    return gradients
```

Listing 1: Norm-based gradient clipping in NumPy.

Gradient clipping limits the update step size to an absolute radius c, preventing sudden parameter jumps when descending steep cliffs.

4 Long Short-Term Memory (LSTM): Engineering an Additive Error Highway

To overcome the vanishing gradient problem, Sepp Hochreiter and Jürgen Schmidhuber (1997) introduced the **Long Short-Term Memory (LSTM)** architecture.

The breakthrough of LSTM lies in separating internal memory from external state, using two paths:

1. **The Cell State ($\mathbf{C}_t \in \mathbb{R}^H$):** Acts as a dedicated, low-resistance linear memory highway running straight down the computational chain.

2. **The Hidden State ($\mathbf{h}_t \in \mathbb{R}^H$):** Acts as short-term working memory, exposed to downstream task heads and gating computations.

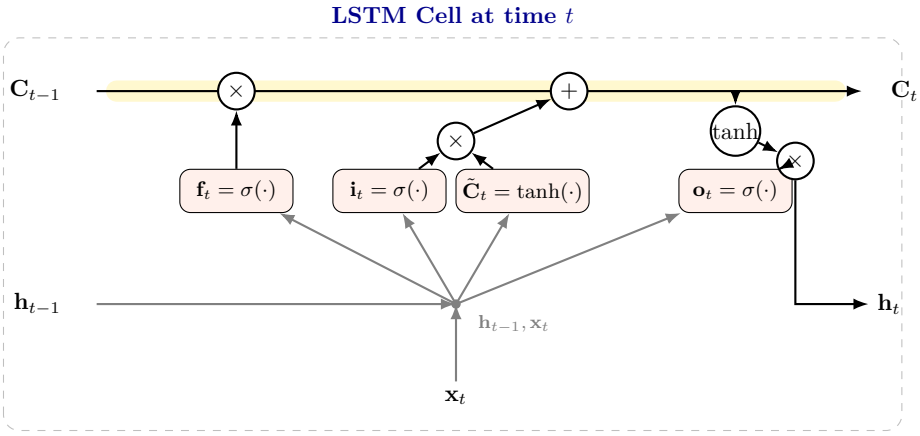


Figure 6: The LSTM cell. The cell state $\mathbf{C}_t$ runs along the top as a protected additive highway (yellow). Three sigmoid gates ($\mathbf{f}_t$, $\mathbf{i}_t$, $\mathbf{o}_t$) modulate what is forgotten, written, and exposed.

The flow of information into and out of the cell state is controlled by three non-linear **gates** parameterized with logistic sigmoid activations ($\sigma \in (0, 1)$):

4.0.1 1. The Forget Gate ($\mathbf{f}_t$)

Decides what portion of historical memory to discard from the cell state based on the current context:

$$\mathbf{f}_t = \sigma(\mathbf{W}_f[\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_f) \tag{21}$$

When $\mathbf{f}_{t,i} \approx 0$, the i-th memory dimension is erased; when $\mathbf{f}_{t,i} \approx 1$, it is preserved entirely.

4.0.2 2. The Input Gate ($\mathbf{i}_t$) and Candidate Cell State ($\tilde{\mathbf{C}}_t$)

Determines what new information to write into long-term memory:

$$\mathbf{i}_t = \sigma(\mathbf{W}_i[\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_i) \tag{22}$$

$$\tilde{\mathbf{C}}_t = \tanh(\mathbf{W}_c[\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_c) \tag{23}$$

- $\mathbf{i}_t$ acts as a soft input filter, selecting which coordinates are worth writing.
- $\tilde{\mathbf{C}}_t$ computes the candidate information values bounded within $(-1, 1)$.

4.0.3 3. Updating the Cell State ($\mathbf{C}_t$)

The cell state update is fundamentally **additive**:

$$\mathbf{C}_t = \mathbf{f}_t \odot \mathbf{C}_{t-1} + \mathbf{i}_t \odot \tilde{\mathbf{C}}_t \tag{24}$$

Where $\odot$ represents the Hadamard (elementwise) product.

4.0.4 4. The Output Gate ($\mathbf{o}_t$) and Final Hidden State ($\mathbf{h}_t$)

Controls what information from the updated cell state is surfaced into the short-term working state $\mathbf{h}_t$:

$$\mathbf{o}_t = \sigma(\mathbf{W}_o[\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_o) \tag{25}$$

$$\mathbf{h}_t = \mathbf{o}_t \odot \tanh(\mathbf{C}_t) \tag{26}$$

4.1 Why the Cell State Prevents Vanishing Gradients

Compute the derivative of the current cell state with respect to the prior state:

$$\frac{\partial \mathbf{C}_t}{\partial \mathbf{C}_{t-1}} = \mathbf{f}_t \tag{27}$$

In an unrolled sequence of T steps, the gradient propagation across the cell highway contains an uninterrupted additive chain:

$$\frac{\partial \mathbf{C}_T}{\partial \mathbf{C}_k} = \prod_{j=k+1}^{T} \mathbf{f}_j \tag{28}$$

If the network learns to keep the forget gates active ($\mathbf{f}_j \approx 1$), **the gradient flows backward across time without exponential decay**. The error does not have to squeeze through repeated matrix multiplications or saturating tanh activations at every step.

4.2 LSTM Parameter Counting

An LSTM computes four distinct affine projections $(\mathbf{f}_t, \mathbf{i}_t, \tilde{\mathbf{C}}_t, \mathbf{o}_t)$ over the concatenated input $[\mathbf{h}_{t-1}, \mathbf{x}_t] \in \mathbb{R}^{H+D}$:

$$\text{Parameters} = 4 \times ((D + H) \times H + H) = 4H(D + H + 1) \tag{29}$$

- For $D = 32$ and $H = 5$:

$$\text{Parameters} = 4 \times 5 \times (32 + 5 + 1) = 20 \times 38 = 760 \text{ parameters} \tag{30}$$

An LSTM requires exactly four times the parameters of a SimpleRNN with equivalent hidden capacity.

5 The Gated Recurrent Unit (GRU): Streamlined Gating

In 2014, Kyunghyun Cho et al. introduced the **Gated Recurrent Unit (GRU)**, simplifying the LSTM mechanics into an efficient two-gate architecture.

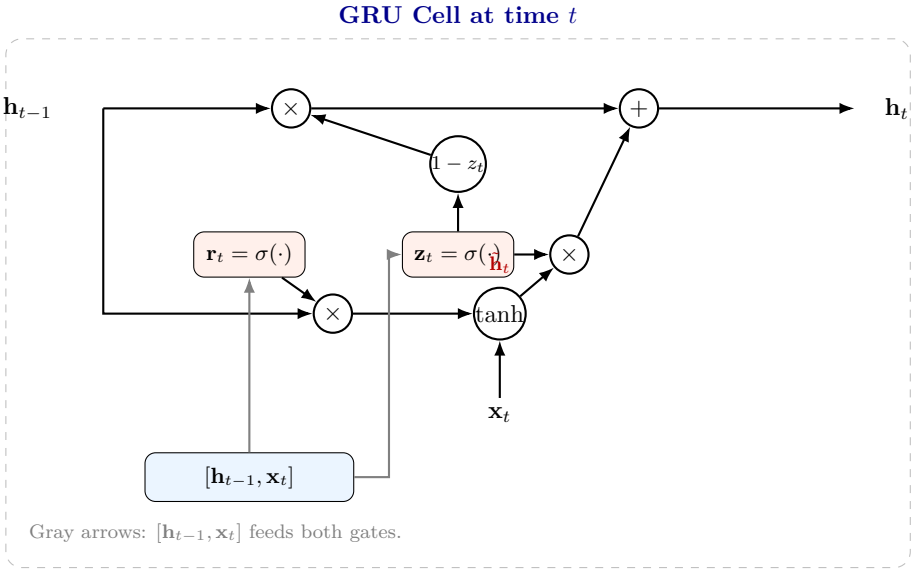


Figure 7: The GRU cell. Two gates ($\mathbf{r}_t$, $\mathbf{z}_t$) control a single hidden state that serves as both memory and output. The update gate linearly interpolates between the previous state and a new candidate.

The GRU introduces two structural simplifications over the LSTM:

1. **Elimination of the Separate Cell State:** The hidden state $\mathbf{h}_t$ serves as both the memory carrier and the external output.
2. **Coupled Update Mechanics:** Rather than learning separate forget and input gates, the update gate manages both through a convex linear combination.

5.0.1 1. The Reset Gate ($\mathbf{r}_t$)

Determines how much of the past hidden state to expose when generating the new candidate representation:

$$\mathbf{r}_t = \sigma(\mathbf{W}_r[\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_r) \tag{31}$$

When $\mathbf{r}_t \approx 0$, the unit ignores past history, acting as if reading the first token of a sequence.

5.0.2 2. The Update Gate ($\mathbf{z}_t$)

Acts as a coupled forget/input gate, deciding the balance between retaining old state versus writing new state:

$$\mathbf{z}_t = \sigma(\mathbf{W}_z[\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_z) \tag{32}$$

5.0.3 3. The Candidate Hidden State ($\tilde{\mathbf{h}}_t$)

Computes new candidate values, gated by the reset vector $\mathbf{r}_t$:

$$\tilde{\mathbf{h}}_t = \tanh(\mathbf{W}_h[\mathbf{r}_t \odot \mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_h) \tag{33}$$

5.0.4 4. The Final Hidden State ($\mathbf{h}_t$)

Interpolates smoothly between the historical state $\mathbf{h}_{t-1}$ and the candidate state $\tilde{\mathbf{h}}_t$ via an elementwise convex combination:

$$\mathbf{h}_t = (1 - \mathbf{z}_t) \odot \mathbf{h}_{t-1} + \mathbf{z}_t \odot \tilde{\mathbf{h}}_t \tag{34}$$

If $\mathbf{z}_t = 0$, the state copies forward unchanged ($\mathbf{h}_t = \mathbf{h}_{t-1}$), allowing gradients to propagate backward through time without decay.

5.1 GRU Parameter Counting

Because a GRU computes three affine transformations $(\mathbf{r}_t, \mathbf{z}_t, \tilde{\mathbf{h}}_t)$ rather than four:

$$\text{Parameters} = 3 \times ((D + H) \times H + H) = 3H(D + H + 1) \tag{35}$$

- For $D = 32$ and $H = 5$:

$$\text{Parameters} = 3 \times 5 \times (32 + 5 + 1) = 15 \times 38 = 570 \text{ parameters} \tag{36}$$

A GRU saves roughly 25% of the parameter footprint compared to an equivalent LSTM while offering comparable representational capacity.

6 Stacked (Deep) Recurrent Architectures

A single-layer recurrent network can learn temporal transitions, but its capacity for hierarchical abstraction is limited. Just as deep feedforward networks build abstract feature hierarchies, we can **stack** multiple recurrent layers vertically.

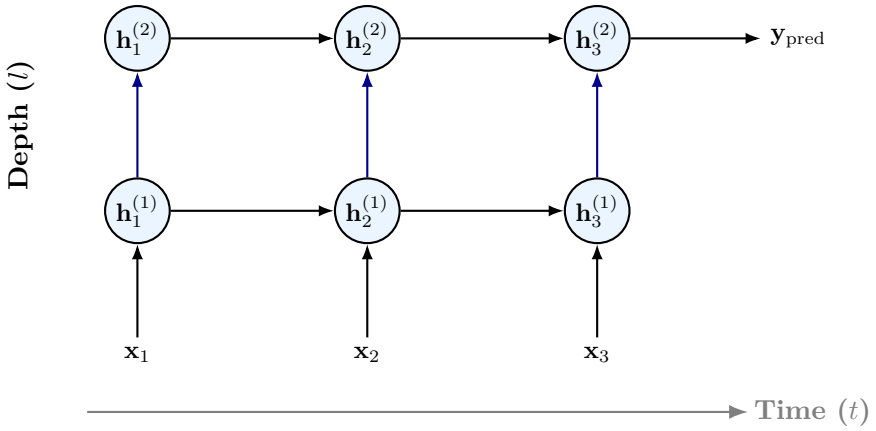


Figure 8: A stacked (deep) recurrent network with two layers. Each layer processes the sequence of hidden states emitted by the layer below.

In a stacked architecture, the sequence of hidden states emitted by layer $l-1$ serves as the input sequence for layer l:

$$\mathbf{h}_t^{(l)} = \text{RNN}^{(l)}\left(\mathbf{h}_t^{(l-1)}, \mathbf{h}_{t-1}^{(l)}\right) \tag{37}$$

6.1 Hierarchical Representation in Language

- **Layer 1 (Surface Syntactics):** Focuses on local transitions, sub-word tokens, and part-of-speech structure.
- **Layer 2 (Phrase-Level Semantics):** Tracks clause boundaries, nominal agreements, and local phrase dependencies.
- **Layer 3+ (Document Context):** Models high-level discourse themes, narrative intent, and document sentiment.

6.2 The Role of `return_sequences`

When chaining recurrent layers, tensor shapes dictate API constraints:

- If Layer l has `return_sequences=False`: It discards intermediate hidden states and outputs only the final vector $\mathbf{h}_T^{(l)} \in \mathbb{R}^H$. Downstream sequence layers cannot process this because the temporal dimension has collapsed.
- If Layer l has `return_sequences=True`: It outputs the full temporal tensor of activations $\mathbf{H}^{(l)} \in \mathbb{R}^{B\times T\times H}$, allowing Layer $l+1$ to process the sequence over all time steps.

7 From Scratch Implementation: A Minimal Next-Word Language Model Engine

Below is a complete, self-contained recurrent pipeline. It includes text sequence generation, an explicit NumPy implementation of forward/backward BPTT dynamics, and Keras functional validation.

```python
import numpy as np

class MinimalRNNLanguageModel:
    """
    A single-layer character/word level recurrent language model engine.
    Implemented from scratch using pure NumPy with explicit forward
    and backward passes unrolled across arbitrary sequence lengths.
    """
    def __init__(self, vocab_size: int, hidden_dim: int = 64, lr: float = 0.01):
        self.V = vocab_size
        self.H = hidden_dim
        self.lr = lr

        # Model parameter tensors
        # Hidden transitions
        self.Wxh = np.random.randn(hidden_dim, vocab_size) * np.sqrt(2.0 / vocab_size)
        self.Whh = np.random.randn(hidden_dim, hidden_dim) * np.sqrt(2.0 / hidden_dim)
        self.bh = np.zeros((hidden_dim, 1))

        # Output projections
        self.Why = np.random.randn(vocab_size, hidden_dim) * np.sqrt(2.0 / hidden_dim)
        self.by = np.zeros((vocab_size, 1))

    def forward(self, inputs: list[int], h_prev: np.ndarray) -> tuple:
        """
        inputs: list of integer vocabulary tokens of length T
        h_prev: initial state vector of shape (H, 1)
        """
        xs, hs, ys, ps = {}, {}, {}, {}
        hs[-1] = np.copy(h_prev)
        loss = 0.0

        for t, idx in enumerate(inputs):
            # One-hot encoding on the fly
            xs[t] = np.zeros((self.V, 1))
            xs[t][idx] = 1.0

            # Recurrence: h_t = tanh(W_xh * x_t + W_hh * h_{t-1} + b_h)
            a_t = np.dot(self.Wxh, xs[t]) + np.dot(self.Whh, hs[t-1]) + self.bh
            hs[t] = np.tanh(a_t)

            # Output: logits and softmax probabilities
            ys[t] = np.dot(self.Why, hs[t]) + self.by
            exp_y = np.exp(ys[t] - np.max(ys[t])) # Numerical stability offset
            ps[t] = exp_y / np.sum(exp_y)

        return xs, hs, ps

    def train_step(self, inputs: list[int], targets: list[int], h_prev: np.ndarray) -> tuple:
        # 1. Forward Pass
        xs, hs, ps = self.forward(inputs, h_prev)
```

```
        # 2. Compute Cross-Entropy Loss
        loss = 0.0
        for t, target_idx in enumerate(targets):
            loss += -np.log(ps[t][target_idx, 0] + 1e-12)

        # 3. Backpropagation Through Time (BPTT)
        dWxh = np.zeros_like(self.Wxh)
        dWhh = np.zeros_like(self.Whh)
        dWhy = np.zeros_like(self.Why)
        dbh = np.zeros_like(self.bh)
        dby = np.zeros_like(self.by)

        dh_next = np.zeros((self.H, 1))

        # Backpropagate backward from step T down to 0
        for t in reversed(range(len(inputs))):
            # Target output gradient: dL/dy = p - y_true
            dy = np.copy(ps[t])
            dy[targets[t]] -= 1.0

            # Gradients into Why and by
            dWhy += np.dot(dy, hs[t].T)
            dby += dy

            # Gradients into hidden state: upstream + recurrent
    contribution
            dh = np.dot(self.Why.T, dy) + dh_next

            # Backprop through tanh: da = dh * (1 - h^2)
            da = dh * (1.0 - hs[t] ** 2)

            # Accumulate weight gradients
            dbh += da
            dWxh += np.dot(da, xs[t].T)
            dWhh += np.dot(da, hs[t-1].T)

            # Propagate back to previous step
            dh_next = np.dot(self.Whh.T, da)

        # 4. Gradient Clipping (Clip values to [-5, 5])
        for grad in [dWxh, dWhh, dWhy, dbh, dby]:
            np.clip(grad, -5.0, 5.0, out=grad)

        # 5. Parameter Updates (SGD)
        self.Wxh -= self.lr * dWxh
        self.Whh -= self.lr * dWhh
        self.Why -= self.lr * dWhy
        self.bh -= self.lr * dbh
        self.by -= self.lr * dby

        return loss, hs[len(inputs) - 1]

    def sample(self, h_init: np.ndarray, seed_idx: int, sample_len: int)
    -> list[int]:
        """
        Autoregressively sample output tokens step-by-step.
        """
        h = np.copy(h_init)
```

```
        x = np.zeros((self.V, 1))
        x[seed_idx] = 1.0

        indices = []
        for _ in range(sample_len):
            h = np.tanh(np.dot(self.Wxh, x) + np.dot(self.Whh, h) + self.bh)
            y = np.dot(self.Why, h) + self.by
            p = np.exp(y - np.max(y)) / np.sum(np.exp(y - np.max(y)))

            # Sample next token from the probability distribution
            idx = np.random.choice(range(self.V), p=p.ravel())
            indices.append(idx)

            # Feed prediction as the next input
            x = np.zeros((self.V, 1))
            x[idx] = 1.0

        return indices
```

Listing 2: A minimal recurrent language model engine implemented from scratch in NumPy.

7.1 Production Verification with Stacked Keras Models

To build deep recurrent models using modern high-level frameworks, we chain stacked layers using `return_sequences=True`:

```
import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Embedding, LSTM, Dense, Dropout

def build_stacked_lstm_predictor(vocab_size: int, embed_dim: int, max_seq_len: int) -> tf.keras.Model:
    model = Sequential([
        # 1. Embedding Layer: maps token indices to dense vectors
        Embedding(input_dim=vocab_size, output_dim=embed_dim, input_length=max_seq_len - 1),

        # 2. Deep LSTM Stack
        # return_sequences=True is mandatory to pass a temporal sequence to the next layer
        LSTM(128, return_sequences=True),
        Dropout(0.2),

        # Intermediate sequence layer
        LSTM(64, return_sequences=False),
        Dropout(0.2),

        # 3. Softmax Head over Vocabulary
        Dense(vocab_size, activation='softmax')
    ])

    model.compile(
        optimizer='adam',
        loss='categorical_crossentropy',
        metrics=['accuracy']
    )
    return model
```

Listing 3: Stacked LSTM predictor in Keras.

8 Architectural Comparison and Diagnostic Guide

Architecture	Gates	State Path	Params / Cell	Best Use Cases
SimpleRNN	0	Single saturating (tanh)	$H(D+H+1)$	Short sequences ($T < 15$); Low-latency embedded microcontrollers.
LSTM	3	Coupled Additive Highway ($\mathbf{C}_t$) + Working State ($\mathbf{h}_t$)	$4H(D+H+1)$	Long sequences (T up to $\sim$500); Complex syntax; High-precision NLP.
GRU	2	Convex combination of hidden state ($\mathbf{h}_t$)	$3H(D+H+1)$	Medium sequences; Constrained compute; Streaming sequences; Comparable to LSTM.

Table 1: Architectural comparison of SimpleRNN, LSTM, and GRU.

8.1 Troubleshooting Sequence Models

Symptom	Mechanical Root Cause	Resolution Protocol
Loss produces `NaN` after a few update iterations	Exploding gradients during long BPTT unrolling	1. Enforce gradient clipping (`clipnorm=1.0`); 2. Switch from SimpleRNN to LSTM or GRU; 3. Lower learning rate by an order of magnitude.
Model outputs the same word monotonically ("the the the...")	Training underflow / local minimum trap	1. Check output layer: verify Softmax is applied; 2. Ensure target labels are properly offset; 3. Add temperature scaling during sampling.
Validation loss diverges while training loss plummets	Overfitting on token sequences due to over-parameterization	1. Introduce Dropout on recurrent inputs and states; 2. Tie embedding weights with output projections; 3. Apply pre-padding instead of post-padding.
Severe training latency on long text sequences	Recurrent steps executed sequentially; no parallelization across time axis	1. Increase batch size to leverage GPU parallelism; 2. Reduce sequence length using sliding chunk windows; 3. Consider migrating to non-recurrent Transformer blocks.

Table 2: Troubleshooting guide for recurrent sequence models.

Exercises

1. **Analytical BPTT for Scalar Sequence:** Consider a toy scalar RNN with no inputs and no bias: $h_t = \tanh(w \cdot h_{t-1})$. Let $h_0 = 1.0$ and $w = 2.0$. Compute numerical values for h_1 and h_2. Derive an analytical expression for $\frac{\partial h_2}{\partial w}$, and evaluate it numerically.

2. **Coupled Input-Forget Gates:** In a classic LSTM variant, the input gate is coupled to the forget gate: $\mathbf{i}_t = 1 - \mathbf{f}_t$. Show how this reduces parameter count, and describe its functional relationship to the update step in a GRU.

3. **Weight Initialization for Forget Gates:** Early LSTM implementations often suffered from slow initial training because the forget gate biases were initialized to 0, causing $\mathbf{f}_t \approx \sigma(0) = 0.5$ and attenuating memory across steps. Prove why initializing forget gate biases to a positive constant (e.g., $b_f = 1.0$ or 2.0) preserves long-term memory early in training.